Computer Networking

Complete Notes v2.0: from zero to CCNA 200-301 (v1.1) and beyond

Fundamentals · OSI & TCP/IP · Cabling · Ethernet · IPv4 & IPv6 Subnetting
VLANs · STP · EtherChannel · Routing · OSPF · FHRP
IP Services · QoS · Wireless & WLC · Security · WAN · SDN · REST/JSON · AI · Cloud

Thirupathi Keloth

100-Day Learning Program · 2026

Contents

1. Network Roles & Career Paths

2. Network Fundamentals

3. OSI & TCP/IP Models

4. Physical Layer: Cabling & Interfaces

5. Ethernet, MAC Addresses & Switching

6. Binary, IPv4 Addressing & Subnetting

7. IPv6

8. Cisco IOS CLI Essentials

9. VLANs & Trunking

10. Spanning Tree Protocol (STP)

11. EtherChannel (Link Aggregation)

12. Routing

13. Transport & Application Protocols

14. IP Services

15. Wireless (Wi-Fi)

16. Network Security Fundamentals

17. WAN & Internet Connectivity

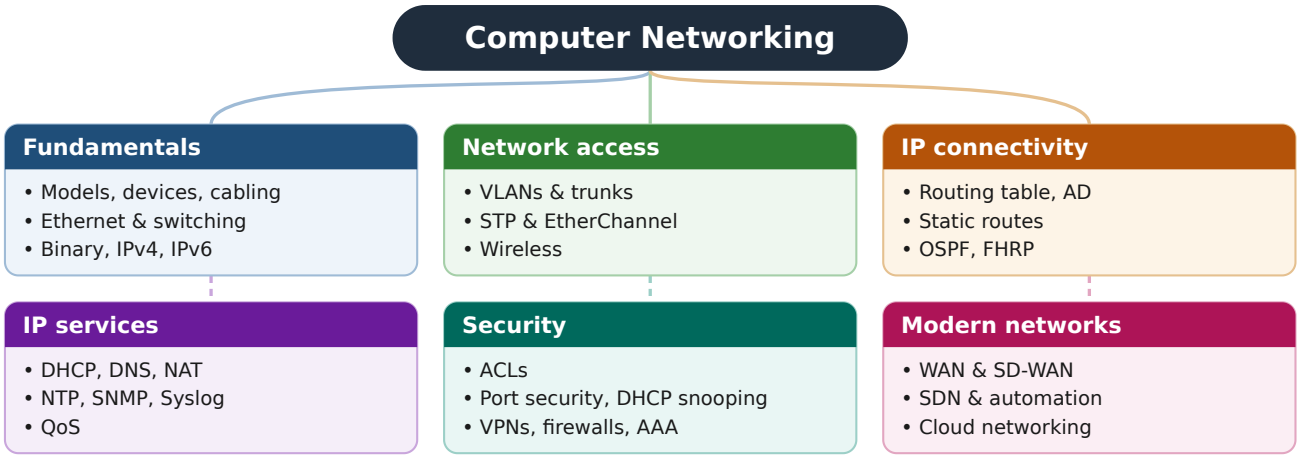
18. Data Center, Virtualization, SDN & Automation

19. Cloud Networking (AWS mapping)

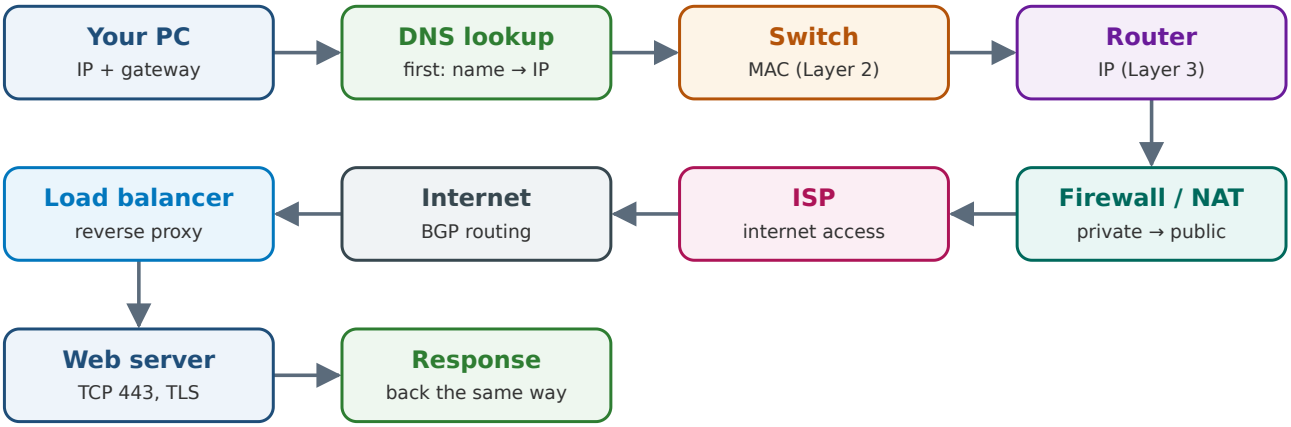
20. Troubleshooting

21. Interview & Exam Preparation

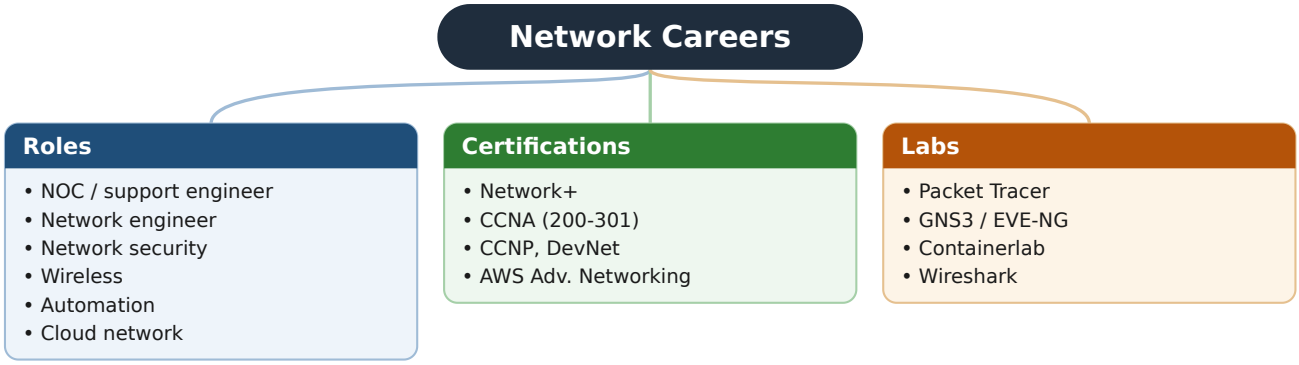
The Big Picture



How a packet travels to a website



1. Network Roles & Career Paths



1.1 Who works on networks

Role	What they do every day	Key skills
Network Support / NOC Engineer	Monitors the network 24/7, handles alerts and tickets, first-level troubleshooting, escalations	Monitoring tools, ping/traceroute, basic switching and routing, ticketing discipline
Network Engineer	Designs, configures and maintains switches, routers, firewalls, Wi-Fi and WAN links; change windows; capacity	VLANs, STP, OSPF/BGP, NAT, ACLs, VPN, Cisco IOS, documentation
Network Security Engineer	Firewalls, VPNs, segmentation, intrusion prevention, network access control	ACLs, NGFW policies, IPsec, 802.1X, zero trust, logging
Wireless Engineer	Designs and tunes Wi-Fi coverage, capacity and security	802.11, RF, WLC, WPA3, site surveys
Network Automation Engineer	Automates changes, checks and inventory with code	Python, Ansible, REST APIs, NETCONF/RESTCONF, Git, CI
Cloud Network Engineer	Builds networks in AWS/Azure/GCP and hybrid links to offices	VPC, subnets, routing, VPN, Direct Connect, Transit Gateway, DNS
DevOps / SRE / Platform Engineer	Uses networking to run applications: DNS, load balancers, TLS, firewalls, Kubernetes networking	TCP/IP, DNS, HTTP, troubleshooting, cloud networking

Networking is the foundation of every infrastructure role: most production incidents eventually involve DNS, routing, firewalls, TLS or load balancers.

1.2 Certifications (in typical order)

Certification	Level	Covers
CompTIA Network+	Entry, vendor-neutral	Networking basics
Cisco CCNA (200-301)	Associate, the industry standard	Everything in these notes

Certification	Level	Covers
AWS Certified Advanced Networking – Specialty	Cloud networking	VPC design, hybrid connectivity, DNS, security
Cisco CCNP Enterprise / Security	Professional	Advanced routing, switching, wireless, SD-WAN, automation / firewalls, VPN
Cisco CCNA Automation (formerly DevNet Associate)	Automation	Network APIs, Python, automation

The CCNA 200-301 exam (version 1.1)

Domain	Weight	Chapters in these notes
1.0 Network Fundamentals	20%	2–7, 15
2.0 Network Access	20%	9–11, 13, 15
3.0 IP Connectivity	25%	12
4.0 IP Services	10%	14
5.0 Security Fundamentals	15%	16
6.0 Automation and Programmability	10%	18

Format: about 120 minutes, multiple choice, drag-and-drop, simulations and simlets (configure or troubleshoot devices in a simulator). Hands-on practice with the CLI is essential; reading alone is not enough.

1.3 Practice labs (free)

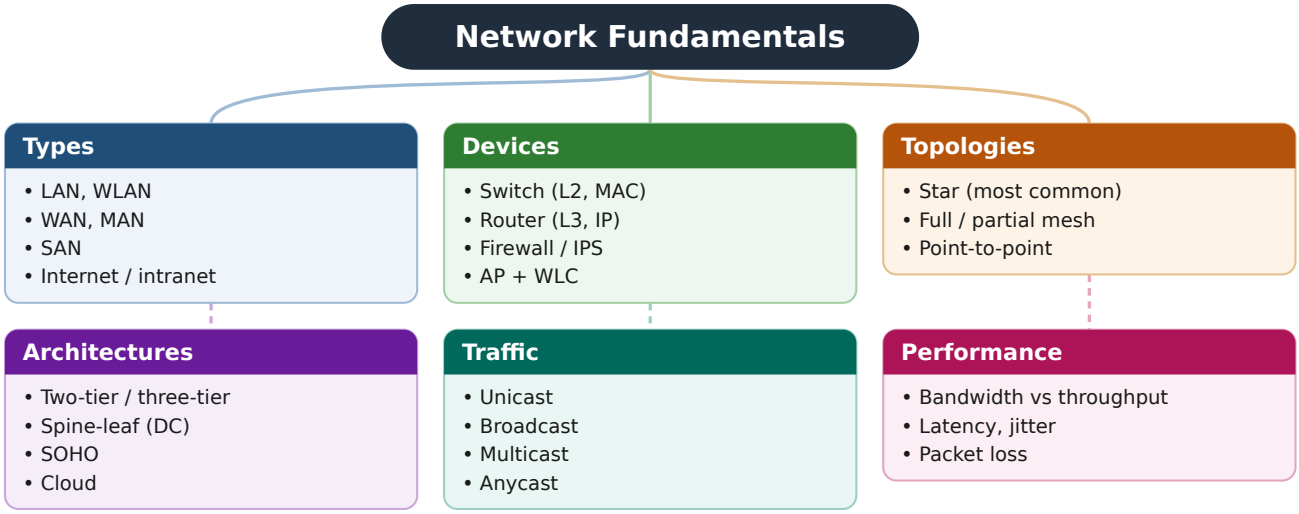
Tool	Use
Cisco Packet Tracer	Simulated Cisco routers/switches/WLCs. Best for CCNA (free with a Cisco Networking Academy account)
GNS3 / EVE-NG / Cisco Modeling Labs (CML Free)	Emulate real network operating systems
Containerlab	Network labs with containers (automation-friendly, open-source NOS images)
Wireshark	Capture and read real packets: the best way to *see* protocols
Your own PC and home router	`ipconfig`, `ping`, `tracert`, `nslookup`, Wireshark on your own traffic

How to study

1. Read one chapter, then **build it in Packet Tracer** the same day.
2. Capture the protocol in Wireshark (ARP, DHCP, DNS, TCP handshake, OSPF hellos).
3. Break your lab on purpose and fix it: that is how troubleshooting skill is built.
4. Do subnetting drills daily for 10 minutes until they're automatic.

> **REMEMBER**
>
> - CCNA domains: Fundamentals · Network Access · IP Connectivity · IP Services · Security · Automation.

2. Network Fundamentals



2.1 What a network is

A **network** is two or more devices connected to share data and resources. Devices that send and receive data are **hosts/endpoints** (PCs, laptops, phones, printers, IP phones, cameras, IoT sensors). **Servers** provide services to clients (web, file, DNS, DHCP, email, databases). Devices that move data between hosts are **intermediary devices** (switches, routers, firewalls, access points).

Client–server vs peer-to-peer: in client-server, dedicated servers provide services; in peer-to-peer, each device can be both client and server (small home networks, file sharing).

2.2 Network types

Type	Meaning	Example
LAN	Local Area Network: one building or campus, owned by the organization	Office floor
WLAN	Wireless LAN	Office Wi-Fi
WAN	Wide Area Network: connects LANs over long distances, usually using a provider	Head office ↔ branches
MAN	Metropolitan Area Network: across a city	City-wide fibre
SAN	Storage Area Network: servers ↔ storage	Data center storage (Fibre Channel, iSCSI)
PAN	Personal Area Network	Bluetooth headset
Internet	Global network of networks connected by ISPs	—
Intranet / Extranet	Private company network / partly opened to partners	Employee portal / supplier portal

2.3 Network devices and their roles

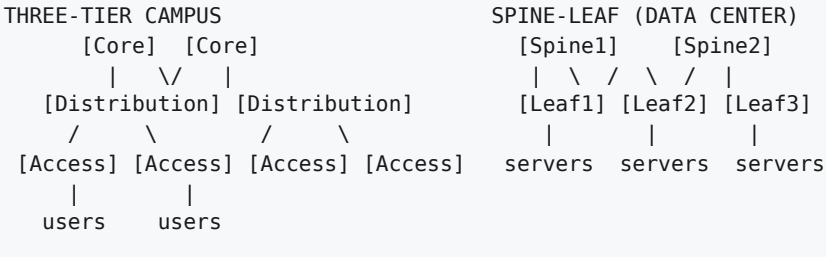
Device	Layer	Job
Hub (obsolete)	1	Repeats bits to every port. One collision domain, half duplex
Switch (Layer 2)	2	Forwards frames by MAC address inside a LAN; separates collision domains; supports VLANs
Multilayer switch (Layer 3)	2 + 3	Switching plus routing between VLANs at hardware speed
Router	3	Forwards packets between networks by IP address; separates broadcast domains; connects to WAN/internet; NAT, VPN
Firewall	3–4	Permits/denies traffic by rules; **stateful** firewalls track connections
Next-Generation Firewall (NGFW)	3–7	Adds application identification, user identity, IPS, URL filtering, malware/TLS inspection
IPS / IDS	3–7	IDS detects and alerts (copy of traffic); IPS sits inline and blocks attacks
Access Point (AP)	1–2	Connects wireless clients to the wired LAN (bridges 802.11 to 802.3)
Wireless LAN Controller (WLC)	—	Centrally manages many lightweight APs: configuration, RF, roaming, security
Load balancer	4–7	Spreads traffic across servers, health checks, TLS termination
Modem / ONT	1	Converts the ISP line (DSL, cable, fibre) to Ethernet
PoE switch	—	Powers phones, APs, cameras over the Ethernet cable (see 4.5)
Endpoints	—	Hosts that create/consume data
Servers	—	Hosts that provide services

2.4 Topologies

Topology	Description	Note
Star	All devices connect to a central switch	Most common LAN design; a central failure affects all
Extended star	Stars connected together (access switches to distribution)	Campus networks
Full mesh	Every device connects to every other	Highly redundant, n(n−1)/2 links: expensive
Partial mesh	Some redundant links	Common in WANs and cores
Bus / Ring	Shared cable / loop	Legacy
Point-to-point	One link between two devices	Router-to-router and WAN links
Hub-and-spoke	Branches connect to a central hub	WAN design: simple, hub is a single point of failure
Hybrid	Mix of the above	Real networks

Physical vs logical topology: physical = how cables actually run; logical = how data flows (VLANs, routing, tunnels).

2.5 Network architectures



Architecture	Where	Idea
Two-tier (collapsed core)	Small/medium campus	Core and distribution combined into one layer
Three-tier	Large campus	**Access** (end devices, VLANs, PoE, port security) → **Distribution** (aggregation, inter-VLAN routing, policy, redundancy) → **Core** (fast, highly available backbone, no policy)
Spine-leaf	Data centers	Every leaf connects to every spine, leaves never connect to each other, spines never connect to each other: every server is the same number of hops away; scale out by adding leaves/spines; carries heavy east-west (server-to-server) traffic
SOHO	Small office/home office	One device = router + switch + Wi-Fi + firewall + modem
WAN	Companies with sites	Hub-and-spoke, full/partial mesh, SD-WAN (chapter 17)
On-premises vs cloud	—	Own equipment in your building/data center (capital cost, full control) vs provider-run networks (VPC/VNet, pay per use, elastic)

North-south vs east-west traffic: north-south enters/leaves the data center (users to servers); east-west flows between servers inside it (microservices, databases, storage), which is why spine-leaf replaced three-tier in data centers.

2.6 Traffic types

Type	Sent to	Example
Unicast	One device	Web browsing
Broadcast	All devices in the local network (`FFFF.FFFF.FFFF` , `255.255.255.255`)	ARP request, DHCP Discover
Multicast	A group of interested devices (`224.0.0.0/4` , `ff00::/8`)	OSPF hellos, video streaming
Anycast	The nearest of several devices with the same address	DNS root servers, CDNs

2.7 Bandwidth, throughput, latency

Term	Meaning
Bandwidth	Maximum capacity of a link (e.g. 1 Gbps)

Term	Meaning
Throughput	Actual data rate achieved (always less: overhead, congestion)
Goodput	Useful application data delivered (excluding headers and retransmissions)
Latency	Time for data to travel (ms). Round-trip time (RTT) = there and back
Jitter	Variation in latency: bad for voice/video
Packet loss	% of packets that never arrive

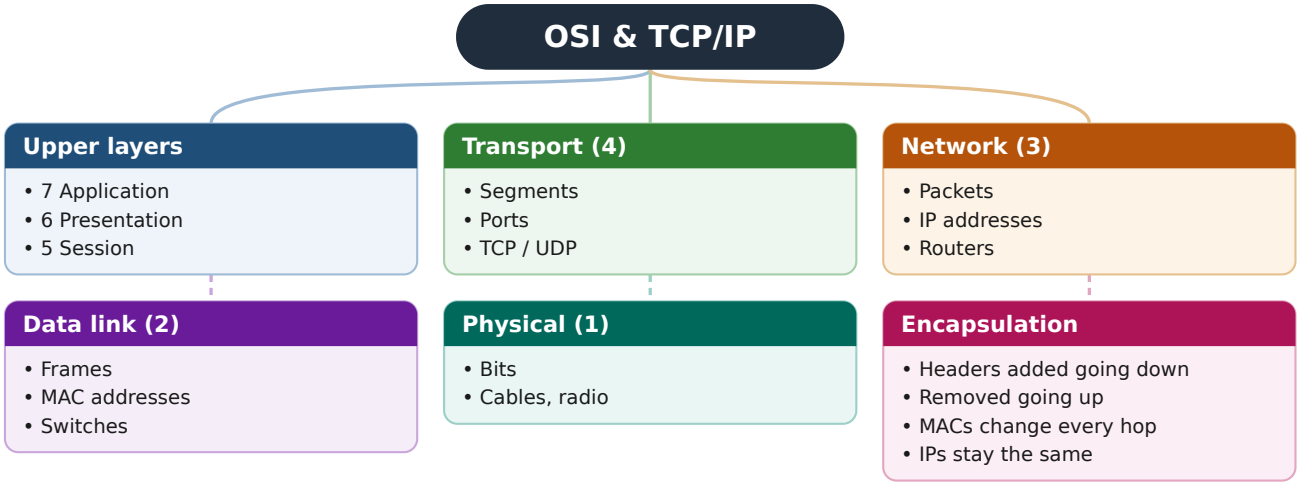
Delay components: **propagation** (distance: ~5 µs per km in fibre), **serialization** (putting bits on the wire), **queuing** (waiting in buffers: the variable part), **processing** (device lookup).

Units: **b = bit, B = byte (8 bits)**. Link speeds are in bits per second (Mbps, Gbps); file sizes usually in bytes (MB). A 100 MB file on a 100 Mbps link takes at least 8 seconds.

REMEMBER

- Three-tier campus: **Access** (users) → **Distribution** (policy) → **Core** (fast backbone).
- Links are measured in **bits** (Mbps); files in **bytes** (MB). 1 byte = 8 bits.

3. OSI & TCP/IP Models



3.1 Why layered models

Layers split networking into independent jobs with standard interfaces: a web browser doesn't care whether the link is Wi-Fi or fibre. Models give engineers a **shared language** ("it's a Layer 2 problem") and a **troubleshooting order**.

3.2 The layers

#	OSI layer	Job	Data unit (PDU)	Address / key info	Protocols / devices
7	Application	Network services for applications	Data	—	HTTP, DNS, SSH, SMTP, FTP, SNMP, DHCP
6	Presentation	Data format, encoding, encryption, compression	Data	—	TLS, JPEG, ASCII/UTF-8
5	Session	Open/manage/close sessions between applications	Data	—	RPC, sockets, NetBIOS
4	Transport	End-to-end delivery between processes; segmentation, reliability, flow control	**Segment** (TCP) / **Datagram** (UDP)	**Port numbers**	TCP, UDP
3	Network	Logical addressing and routing between networks	**Packet**	**IP address**	IPv4, IPv6, ICMP, OSPF; routers
2	Data link	Delivery on the local link; framing, MAC addressing, error detection	**Frame**	**MAC address**	Ethernet, 802.11, ARP, PPP; switches, APs
1	Physical	Bits as signals on the medium	**Bits**	Signals, voltages, light, radio	Cables, connectors, transceivers, hubs

Memory trick (7 → 1): **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.

TCP/IP model (what the internet actually uses): Application (OSI 5–7) · Transport (4) · Internet (3) · Link / Network Access (1–2). Some books split Link into Data Link and Physical (5-layer model).

3.3 Encapsulation

```
Sender (going down)                        Receiver (going up)
Data                                       Data
[TCP hdr | Data]            = Segment      ↑ remove TCP header (port → which application)
[IP hdr | TCP hdr | Data]   = Packet       ↑ remove IP header (is it for me?)
[Eth hdr | IP | TCP | Data | FCS] = Frame ↑ check FCS, remove Ethernet header
010110101...                = Bits         ↑
```

- **Encapsulation:** each layer adds its header (and Ethernet adds a trailer, the FCS) on the way down. **De-encapsulation:** removed on the way up.
- **Same-layer interaction:** each layer "talks" to the same layer on the other device through its header. **Adjacent-layer interaction:** each layer provides a service to the layer above.
- **Key rule:** while a packet crosses networks, the **IP addresses stay the same** end to end (unless NAT changes them), but the **MAC addresses change at every router hop**.

Worked example: a packet crossing two routers

```
PC-A 10.1.1.10 (MAC AAAA) ── SW ── R1 g0/0 10.1.1.1 (MAC R1A) | R1 g0/1 ── R2 ── Server 10.2.2.20
                                                                (MAC R1B)  (MAC R2A)  (MAC SSSS)
Hop 1, PC-A → R1:  Src MAC AAAA  Dst MAC R1A   | Src IP 10.1.1.10  Dst IP 10.2.2.20
Hop 2, R1 → R2:    Src MAC R1B   Dst MAC R2A   | Src IP 10.1.1.10  Dst IP 10.2.2.20  (TTL − 1)
Hop 3, R2 → Srv:   Src MAC R2B   Dst MAC SSSS  | Src IP 10.1.1.10  Dst IP 10.2.2.20  (TTL − 1)
```

PC-A sees the destination is in another network (by comparing with its own mask), so it sends the frame to its **default gateway's MAC** (learned with ARP). Each router strips the old frame, looks up the destination IP in its routing table, decrements TTL, and builds a new frame for the next hop.

3.4 Important headers

Ethernet II frame

Field	Size	Meaning
Preamble + SFD	7 + 1 bytes	Synchronization; Start Frame Delimiter (not counted in frame size)
Destination MAC	6 bytes	Receiver (unicast, multicast, broadcast)
Source MAC	6 bytes	Sender (always unicast)
802.1Q tag (optional)	4 bytes	VLAN ID and priority on trunks (chapter 9)
Type	2 bytes	Payload protocol: `0x0800` IPv4, `0x86DD` IPv6, `0x0806` ARP, `0x8100` VLAN tag
Payload	46–1500 bytes	The packet. **MTU = 1500** bytes by default; padded to 46 if smaller
FCS	4 bytes	Frame Check Sequence (CRC) to detect errors; bad frames are dropped

Minimum frame 64 bytes, maximum 1518 bytes (1522 with a VLAN tag). Jumbo frames (up to ~9000 bytes) are used in data centers.

IPv4 header (20 bytes without options)

Field	Meaning
Version	4
IHL	Header length (in 4-byte words; 5 = 20 bytes)
DSCP / ECN	QoS marking (chapter 14) / congestion notification
Total length	Header + data, max 65,535
Identification, Flags, Fragment offset	Fragmentation (DF = "don't fragment" flag)
TTL	Decreases by 1 at each router; at 0 the packet is dropped and ICMP "time exceeded" is sent (prevents endless loops; traceroute uses it)
Protocol	Payload: 1 = ICMP, 6 = TCP, 17 = UDP, 89 = OSPF, 50 = ESP
Header checksum	Error check of the header (recomputed at each hop because TTL changes)
Source / Destination IP	32 bits each

TCP and UDP headers

TCP field	Meaning
Source / destination port	Identify the applications (client uses an ephemeral port)
Sequence number	Position of this data in the byte stream
Acknowledgment number	Next byte expected from the other side
Flags	SYN (start), ACK, FIN (finish), RST (abort), PSH, URG
Window	How many bytes the receiver can accept now (flow control)
Checksum	Error detection

UDP header: source port, destination port, length, checksum: only **8 bytes**. Details of TCP behavior are in chapter 13.

3.5 ICMP

ICMP (Internet Control Message Protocol, IP protocol 1) carries errors and diagnostics:

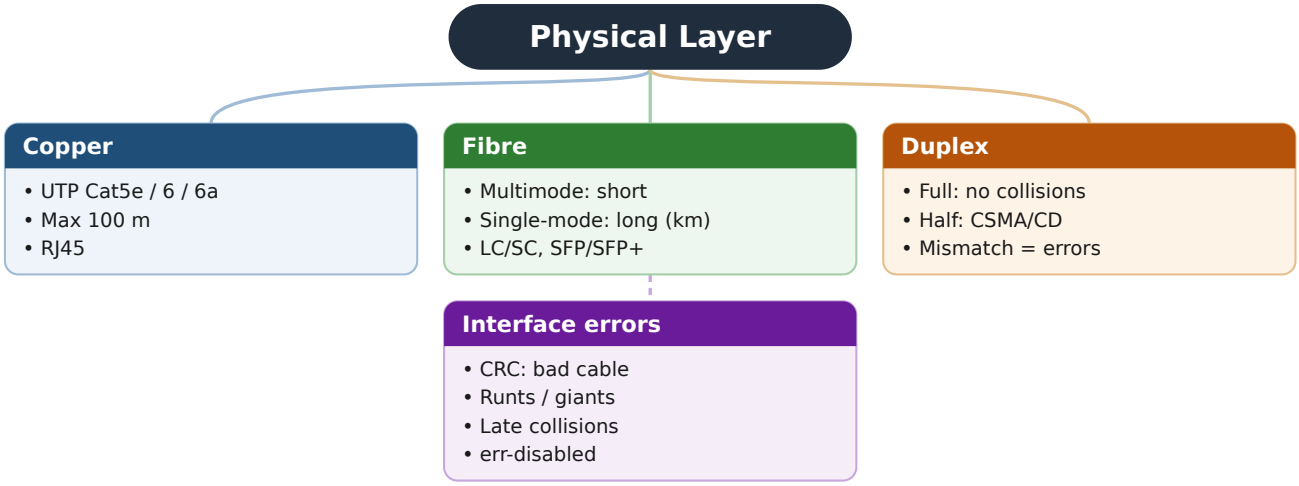
Type	Name	Used by
8 / 0	Echo request / echo reply	`ping`
3	Destination unreachable (codes: network, host, port, fragmentation needed)	Routers/hosts reporting failures; Path MTU discovery
11	Time exceeded (TTL reached 0)	`traceroute`
5	Redirect	Telling a host about a better gateway

How traceroute works: it sends packets with TTL 1, 2, 3…; each router where TTL hits 0 replies with "time exceeded", revealing itself. Windows `tracert` uses ICMP; Linux/Cisco `traceroute` use UDP by default.

REMEMBER

- 7 → 1: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing. 1 → 7: **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way.
- PDUs: **Data → Segment → Packet → Frame → Bits**.

4. Physical Layer: Cabling & Interfaces



4.1 Copper cabling

Cable	Max distance	Typical speed	Notes
UTP Cat5e	100 m	1 Gbps	Minimum for modern LANs
UTP Cat6	100 m (10 Gbps up to ~55 m)	1–10 Gbps	Common in new buildings
UTP Cat6a	100 m	10 Gbps	Data centers, Wi-Fi 6/7 AP uplinks
Cat8	30 m	25/40 Gbps	Short data center links
STP / FTP (shielded)	100 m	—	Electrically noisy environments (factories)
Twinax / DAC	1–7 m	10–100 Gbps	Direct-attach copper between switches/servers in a rack

- **UTP** = unshielded twisted pair: 4 twisted pairs; twisting cancels electromagnetic interference (EMI) and crosstalk.
- Connector: **RJ45**. Wiring standards T568A/T568B.
- **Straight-through vs crossover:** historically, unlike devices (PC–switch) used straight-through and like devices (switch–switch) used crossover. **Auto-MDIX** on modern ports detects and adjusts automatically.
- 10/100 Mbps use 2 pairs (pins 1,2,3,6); 1000BASE-T and faster use all 4 pairs.

4.2 Fibre cabling

Type	Core	Light source	Distance	Use
Multimode (MMF)	50 or 62.5 µm (wide)	LED / VCSEL	Up to a few hundred metres (e.g. 10G-SR ≈ 300–400 m on OM3/OM4)	Inside buildings, data centers

Type	Core	Light source	Distance	Use
Single-mode (SMF)	~9 µm (narrow)	Laser	Kilometres (10G-LR 10 km, ER 40 km)	Between buildings, campuses, WAN, ISPs

Fibre advantages: long distance, immune to EMI, more bandwidth, harder to tap. Connectors: **LC** (small, most common), SC, MPO/MTP (multi-fibre, 40/100G). Fibre uses separate transmit and receive strands (or BiDi optics on one strand).

Transceivers plug into switch ports: **SFP** (1G), **SFP+** (10G), SFP28 (25G), **QSFP+** (40G), QSFP28 (100G), QSFP-DD (400G). The optic type (SR, LR) must match the fibre type at both ends.

4.3 Ethernet standards (IEEE 802.3)

Standard	Speed	Media	Max distance
10BASE-T	10 Mbps	Cat3+ copper	100 m
100BASE-TX	100 Mbps (Fast Ethernet)	Cat5 copper	100 m
1000BASE-T	1 Gbps	Cat5e copper	100 m
1000BASE-SX / LX	1 Gbps	Multimode / single-mode fibre	~550 m / 5–10 km
10GBASE-T	10 Gbps	Cat6a copper	100 m
10GBASE-SR / LR	10 Gbps	Multimode / single-mode fibre	~300–400 m / 10 km
25G, 40G, 100G, 400G	—	Mostly fibre/DAC	Data centers and backbones

Naming: number = speed in Mbps (or G), BASE = baseband, suffix = media (T = twisted pair, S = short wavelength, L = long).

4.4 Speed, duplex and auto-negotiation

- **Half duplex:** send or receive, not both (hubs, Wi-Fi). Wired half duplex uses **CSMA/CD** (listen before sending; on collision, jam signal and random back-off).
- **Full duplex:** send and receive simultaneously on separate pairs: no collisions, CSMA/CD disabled. All switch-to-device links today.
- **Auto-negotiation** (IEEE 802.3u/ab): both sides advertise capabilities and pick the best common speed/duplex.
- If one side is hard-coded and the other auto-negotiates: the auto side can sense **speed** but not duplex, so it falls back to **half duplex** at 10/100 Mbps (full at 1 Gbps+) → **duplex mismatch**.
- **Duplex mismatch symptoms:** link up, but slow; **late collisions** on the half-duplex side, **CRC/runts** on the full-duplex side. Fix: auto on both sides or the same fixed settings on both.

```
SW1(config)# interface g0/1
SW1(config-if)# speed 1000
SW1(config-if)# duplex full
SW1(config-if)# description Uplink to R1 g0/0
SW1# show interfaces status                ! port, VLAN, duplex, speed (a-full = auto-negotiated)
```

4.5 Power over Ethernet (PoE)

Standard	Name	Power at PSE (switch port)	Typical device
802.3af	PoE	15.4 W	IP phones, basic cameras
802.3at	PoE+	30 W	Wi-Fi APs, PTZ cameras
802.3bt Type 3	PoE++ / UPoE	60 W	High-end APs, thin clients
802.3bt Type 4	PoE++	90–100 W	Displays, lighting, building systems

PSE = power sourcing equipment (switch or injector); **PD** = powered device. The switch detects a PD before applying power and classifies how much it needs; plan the switch's **total PoE budget** (e.g. 48 ports × 30 W exceeds most switch budgets). Commands: `show power inline`, `power inline auto|never` per interface. **Power policing** can shut down or log a device that draws more than allowed.

4.6 Shared media vs point-to-point

	Shared medium	Point-to-point
Examples	Wi-Fi, old hubs, coax bus	Switch port to host, router-to-router links, serial WAN
Access	Devices contend for the medium (CSMA/CD wired, CSMA/CA wireless)	Dedicated link, full duplex

4.7 Interface status and errors

```
R1# show interfaces g0/1
GigabitEthernet0/1 is up, line protocol is up
  Hardware is ..., address is 001a.2b3c.4d5e
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec
  Full-duplex, 1000Mb/s, media type is RJ45
  5 minute input rate 2000 bits/sec, 3 packets/sec
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     0 output errors, 0 collisions, 0 interface resets
     0 late collision, 0 deferred
```

Status (interface / line protocol)	Meaning
up / up	Working
administratively down / down	Interface is `shutdown` → `no shutdown`
down / down	Layer 1 problem: no cable, cable fault, other side shut down, speed mismatch
up / down	Layer 1 OK but Layer 2 problem: encapsulation mismatch, keepalives (serial links)
down / down (err-disabled)	Shut by a protection feature: port security, BPDU Guard, EtherChannel misconfig (`show interfaces status err-disabled`)

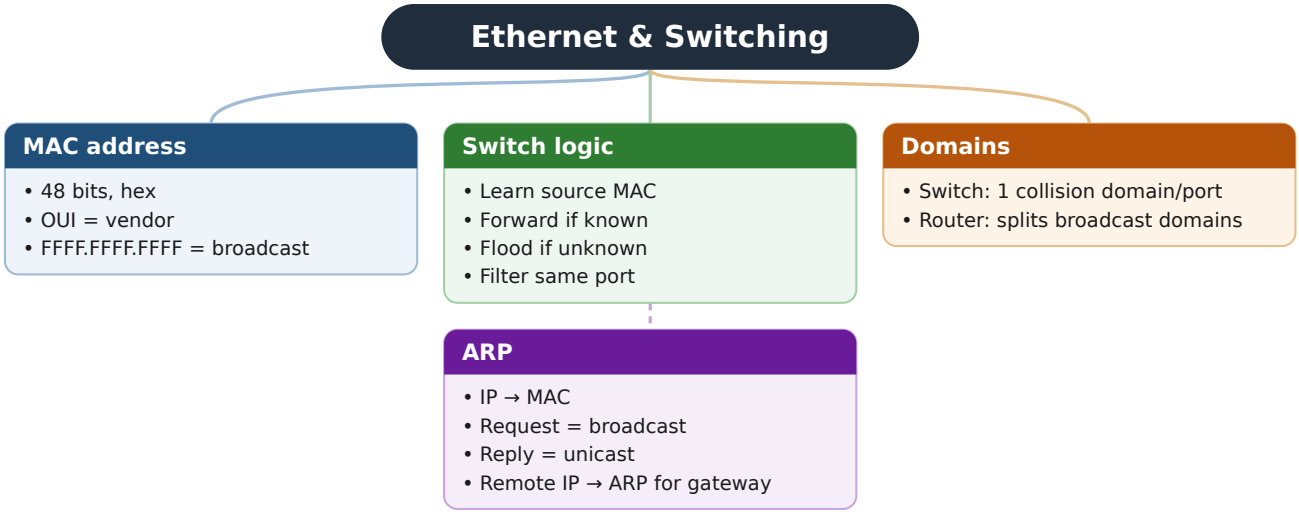
Counter	Meaning / likely cause
CRC / input errors	Corrupted frames: bad cable, interference, bad port or transceiver, duplex mismatch
Runts	Frames < 64 bytes: collisions, duplex mismatch, faulty NIC
Giants	Frames > 1518 bytes: MTU mismatch, jumbo frames where unsupported
Collisions	Normal on half-duplex only; on full-duplex links = mismatch
Late collisions	Collision after the first 64 bytes: **duplex mismatch** or cable too long
Output drops / queue drops	Congested interface (more traffic than the link carries)
Interface resets / flaps	Unstable link: cable, optic, power, remote device rebooting

Troubleshooting order for a dead link: link lights → `show interfaces status` → cable/optic swap → both ends' speed/duplex → `shutdown` / `no shutdown` → err-disabled reason.

REMEMBER

- Copper Ethernet limit: **100 metres**. Long distance → **single-mode fibre**.
- `administratively down` → you forgot `no shutdown`.

5. Ethernet, MAC Addresses & Switching



5.1 MAC addresses

- 48 bits written in hex: `00:1A:2B:3C:4D:5E` (Linux), `00-1A-2B-3C-4D-5E` (Windows), `001a.2b3c.4d5e` (Cisco).
- First 24 bits = **OUI** (Organizationally Unique Identifier: the vendor), last 24 bits = assigned by the vendor.
- Burned into the NIC ("BIA", burned-in address), but software can change it (virtual machines, randomized MACs on phones for privacy).
- **Unicast** (one NIC), **broadcast** `FFFF.FFFF.FFFF` (everyone in the VLAN), **multicast** (first byte's lowest bit = 1, e.g. `0100.5e..` for IPv4 multicast, `3333.....` for IPv6 multicast).

5.2 How a switch forwards frames

```
1. LEARN    : frame arrives on port Fa0/1 from MAC A → table: "A is on Fa0/1 (VLAN 10)"
2. FORWARD  : destination MAC known → send only out that port              (known unicast)
3. FLOOD    : destination unknown, broadcast or multicast → send out all ports in the VLAN
              except the one it came in on                                  (unknown unicast)
4. FILTER   : destination is on the same port it came from → drop
```

- The switch learns from the **source** MAC and forwards based on the **destination** MAC.
- The **MAC address table** (CAM table) maps MAC + VLAN → port. Dynamic entries **age out after 300 seconds** of inactivity (`mac address-table aging-time`).
- If a MAC moves to another port, the entry updates (frequent MAC "flapping" in logs indicates a loop or misconfiguration).
- **Switching methods: store-and-forward** (receive the whole frame, check FCS, then forward: default on most Cisco switches), **cut-through** (forward after reading the destination MAC: lower latency, forwards errored frames; data center switches), fragment-free (after the first 64 bytes).

```
SW1# show mac address-table
          Mac Address Table
-------------------------------------------
Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  10    001a.2b3c.4d5e    DYNAMIC     Fa0/1
  10    0050.7966.6801    DYNAMIC     Gi0/1      <- many MACs on one port = uplink to another switch
  20    0050.7966.6802    STATIC      Fa0/5
SW1# show mac address-table address 001a.2b3c.4d5e
SW1# show mac address-table interface fa0/1
SW1# show mac address-table count
SW1# clear mac address-table dynamic
```

5.3 Collision vs broadcast domains

Device	Collision domains	Broadcast domains
Hub	One for all ports	One
Switch	One **per port**	One per VLAN
Router	One per port	One **per interface** (routers do not forward broadcasts)

Example: a router with 2 interfaces, each connected to a switch with 4 PCs and no VLANs = 2 broadcast domains, 10 collision domains (8 PC ports + 2 router links).

5.4 ARP (Address Resolution Protocol)

Finds the MAC address for a known IPv4 address on the **local** network.

```
PC1 (10.0.0.10) wants to send to 10.0.0.20 (same subnet):
  1. Check ARP cache. Not there →
  2. ARP Request  (broadcast to FFFF.FFFF.FFFF): "Who has 10.0.0.20? Tell 10.0.0.10"
  3. ARP Reply    (unicast):   "10.0.0.20 is at aaaa.bbbb.cccc"
  4. PC1 caches it (Windows ~minutes, Cisco 4 hours) and sends the frame.

Destination in ANOTHER subnet (e.g. 8.8.8.8):
  PC1 ARPs for its DEFAULT GATEWAY's IP and sends the frame to the gateway's MAC.
```

Term	Meaning
ARP cache / table	Stored IP ↔ MAC mappings (`arp -a` Windows, `ip neigh` Linux, `show ip arp` Cisco)
Gratuitous ARP	Unrequested ARP announcing "my IP is at my MAC": detects duplicates, updates others after failover (HSRP, VM moves)
Proxy ARP	A router answers ARP on behalf of hosts in another network (legacy; usually disabled)
ARP spoofing	Attacker sends fake replies to intercept traffic → defense: Dynamic ARP Inspection (16.6)

IPv6 replaces ARP with **Neighbor Discovery** (chapter 7).

5.5 Frame walk-through: two PCs on one switch

1. PC1 wants to ping PC2 (same subnet). ARP request broadcast → switch learns PC1's MAC on Fa0/1, floods the broadcast to all ports in the VLAN.
2. PC2 replies (unicast) → switch learns PC2's MAC on Fa0/2, forwards the reply only to Fa0/1 (known).

3. PC1 sends the ICMP echo request to PC2's MAC → forwarded only to Fa0/2.

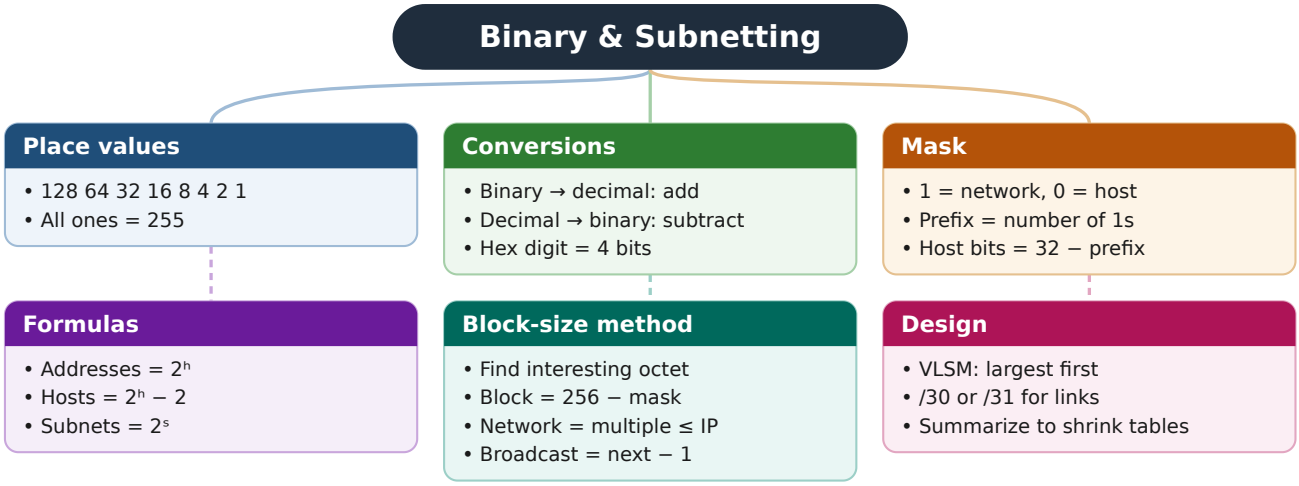
Check your understanding with Packet Tracer simulation mode: watch the MAC table fill in.

5.6 MTU

The **MTU** (maximum transmission unit) is the largest payload a link carries: 1500 bytes on Ethernet by default. Larger packets are **fragmented** (IPv4, if DF is not set) or dropped with ICMP "fragmentation needed" (used by **Path MTU Discovery**). Tunnels (GRE, IPsec, VXLAN) add headers and reduce the effective MTU: a classic cause of "small pings work, large transfers hang". Check with `ping -f -l 1472 host` (Windows) or `ping -M do -s 1472 host` (Linux): 1472 + 28 bytes of IP/ICMP headers = 1500.

> **REMEMBER**
>
> - Switches learn from the **source** MAC and forward by the **destination** MAC.
> - Routers stop broadcasts; switches (and VLANs) contain them.

6. Binary, IPv4 Addressing & Subnetting



6.1 Number systems

System	Base	Digits	Used for
Decimal	10	0–9	How people write IPv4: `192.168.1.10`
Binary	2	0, 1	How computers actually store IPs and masks
Hexadecimal (hex)	16	0–9, A–F	MAC addresses, IPv6

Unit	Size
Bit	One 0 or 1
Nibble	4 bits (= one hex digit)
Byte / **octet**	8 bits. An IPv4 address = 4 octets = 32 bits

6.2 Place values (the key to everything)

Each position in an 8-bit octet has a value. **Memorize this row:**

Position	8th	7th	6th	5th	4th	3rd	2nd	1st
Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**

All 8 bits on: 128+64+32+16+8+4+2+1 = **255** → that's why an octet goes from 0 to 255.

Powers of 2 (for counting addresses):

2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{11}	2^{12}	2^{16}
2	4	8	16	32	64	128	256	512	1024	2048	4096	65536

6.3 Binary → decimal

Write the place values above the bits and **add the values where the bit is 1**.

Binary	128	64	32	16	8	4	2	1	Sum
11000000	1	1	0	0	0	0	0	0	128+64 = **192**
10101000	1	0	1	0	1	0	0	0	128+32+8 = **168**
00001010	0	0	0	0	1	0	1	0	8+2 = **10**
01001101	0	1	0	0	1	1	0	1	64+8+4+1 = **77**

6.4 Decimal → binary (subtraction method)

Go left to right. **If the number is ≥ the place value: write 1 and subtract. Otherwise write 0.**

Example: 172

Place value	128	64	32	16	8	4	2	1
Fits?	172 ≥ 128 ✓	44 < 64 ✗	44 ≥ 32 ✓	12 < 16 ✗	12 ≥ 8 ✓	4 ≥ 4 ✓	0 ✗	0 ✗
Remaining	44	44	12	12	4	0	0	0
Bit	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**0**

172 = **10101100**. Check: 128+32+8+4 = 172 ✓

Example: 200 → 128 ✓ (72 left) → 64 ✓ (8 left) → 32 ✗ → 16 ✗ → 8 ✓ (0 left) → 4 ✗ → 2 ✗ → 1 ✗ = **11001000**

Full address:

```
192      .168      .10       .77
11000000 .10101000 .00001010 .01001101
```

6.5 Hexadecimal

One hex digit = 4 bits. A=10, B=11, C=12, D=13, E=14, F=15.

Hex	Binary	Hex	Binary	Hex	Binary	Hex	Binary
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

- **Hex → decimal:** `1A` = 1×16 + 10 = **26**. `FF` = 15×16 + 15 = **255**. `2F` = 2×16 + 15 = **47**.
- **Hex → binary:** convert each digit separately: `0DB8` = 0000 1101 1011 1000.
- Used in MAC addresses (`00:1A:2B:...`) and IPv6 (`2001:db8::1`).

6.6 The subnet mask in binary

An IPv4 address has a **network part** (which network) and a **host part** (which device). The **subnet mask** marks them: **1 = network bit, 0 = host bit**. The 1s are always continuous from the left.

```
/24  = 11111111.11111111.11111111.00000000 = 255.255.255.0     (24 network bits, 8 host bits)
/27  = 11111111.11111111.11111111.11100000 = 255.255.255.224   (27 network bits, 5 host bits)
```

Prefix length (`/27`) = number of 1s. **Host bits (h)** = 32 − prefix.

Only these values can appear in a mask octet:

Mask octet	Binary	Network bits in octet	Block size (256 − value)
0	00000000	0	256
128	10000000	1	128
192	11000000	2	64
224	11100000	3	32
240	11110000	4	16
248	11111000	5	8
252	11111100	6	4
254	11111110	7	2
255	11111111	8	1

The formulas:

Want to know	Formula	Example /27 (h = 5)
Total addresses in a subnet	2^h	$2^5 = 32$
Usable hosts	$2^h - 2$ (network + broadcast reserved)	$32 - 2 = 30$
Number of subnets when borrowing s bits	2^s	/24 → /27 borrows 3 bits → $2^3 = 8$ subnets

6.7 How the network address is found: the AND operation

Routers find the network by doing a bitwise **AND** of the IP and the mask. **AND rule: 1 AND 1 = 1, anything else = 0.**

Example: 192.168.10.77/27 (only the last octet changes, because the first three mask octets are 255)

```
IP   last octet   77 = 0 1 0 | 0 1 1 0 1
Mask last octet  224 = 1 1 1 | 0 0 0 0 0
                       network | host bits
AND result           = 0 1 0 | 0 0 0 0 0 = 64   → Network   192.168.10.64

Set all host bits to 1 = 0 1 0 | 1 1 1 1 1 = 95   → Broadcast 192.168.10.95
```

Result	Value
Network address (all host bits 0)	192.168.10.64
First usable host (network + 1)	192.168.10.65
Last usable host (broadcast − 1)	192.168.10.94
Broadcast address (all host bits 1)	192.168.10.95
Usable hosts	$2^5 - 2 = 30$

6.8 The fast method: block size

Use this in exams and real work. No binary needed once you understand 6.7.

1. **Find the interesting octet:** the octet where the mask is **not** 255 or 0.

Prefix	/1–/8	/9–/16	/17–/24	/25–/32
Interesting octet	1st	2nd	3rd	4th

2. **Block size** = 256 − mask value in that octet.
3. Subnets start at **multiples of the block size**: 0, block, 2×block...
4. **Network** = the largest multiple that is ≤ the IP's value in that octet. Octets to the right become 0.
5. **Broadcast** = next network − 1 (octets to the right become 255).

Example 1: 10.20.30.200/26

- /26 → 4th octet, mask 192 → block **64** → subnets .0, .64, .128, **.192**
- 200 is in the .192 block → **Network 10.20.30.192**, broadcast **10.20.30.255**, hosts .193 – .254 (62 hosts)

Example 2: 172.16.45.10/20

- /20 → 3rd octet, mask 255.255.**240**.0 → block **16** → 0, 16, **32**, 48...
- 45 is in the 32 block → **Network 172.16.32.0**, broadcast **172.16.47.255**
- Hosts 172.16.32.1 – 172.16.47.254 → $2^{12} - 2$ = **4,094**

Example 3: 192.168.5.130/30

- block 4 → .128, .132 → **Network .128**, hosts **.129, .130**, broadcast **.131** (typical router-to-router link)

Example 4: 10.5.200.9/14

- /14 → 2nd octet, mask 255.**252**.0.0 → block **4** → 0, **4**, 8...
- 5 is in the 4 block → **Network 10.4.0.0**, broadcast **10.7.255.255**, $2^{18} - 2$ = 262,142 hosts

6.9 Designing subnets

A) "I need N subnets" → find the smallest s where $2^s \geq N$, add s to the prefix.

Need 6 subnets from 192.168.1.0/24 → $2^3 = 8 \geq 6$ → borrow 3 bits → **/27** → block 32, 30 hosts each:

#	Network	Usable range	Broadcast
1	192.168.1.0/27	.1 – .30	.31

#	Network	Usable range	Broadcast
2	192.168.1.32/27	.33 – .62	.63
3	192.168.1.64/27	.65 – .94	.95
4	192.168.1.96/27	.97 – .126	.127
5	192.168.1.128/27	.129 – .158	.159
6	192.168.1.160/27	.161 – .190	.191
7–8	.192/27, .224/27	spare	

B) "I need H hosts per subnet" → find the smallest h where $2^h - 2 \geq H$, prefix = $32 - h$.

Hosts needed (up to)	2	6	14	30	62	126	254	510	1022	2046	4094
Prefix	/30	/29	/28	/27	/26	/25	/24	/23	/22	/21	/20

Need 500 hosts → $2^9 - 2 = 510 \geq 500$ → h = 9 → **/23**.

6.10 Address classes (historic) and special ranges

Class	First octet	Default mask	Note
A	1–126	/8	127 = loopback
B	128–191	/16	
C	192–223	/24	
D	224–239	—	Multicast
E	240–255	—	Experimental

Classes are no longer used for routing (we use CIDR), but the terms still appear.

Range	Purpose
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback
169.254.0.0/16	APIPA / link-local (no DHCP answer)
100.64.0.0/10	Carrier-grade NAT (ISP use)
0.0.0.0/0	Default route ("everything")
255.255.255.255	Limited broadcast

6.11 Prefix reference table

Prefix	Mask	Addresses	Usable hosts
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/20	255.255.240.0	4,096	4,094
/22	255.255.252.0	1,024	1,022

Prefix	Mask	Addresses	Usable hosts
/23	255.255.254.0	512	510
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2 (point-to-point links)
/31	255.255.255.254	2	2 (point-to-point, RFC 3021)
/32	255.255.255.255	1	Single host / loopback

6.12 VLSM (Variable Length Subnet Masking)

Give each subnet only the size it needs. **Allocate the largest first**, so the blocks line up.

Example from `192.168.1.0/24`:

Need	Hosts	Prefix	Subnet	Range	Broadcast
Sales	100	/25	192.168.1.0	.1 – .126	.127
IT	50	/26	192.168.1.128	.129 – .190	.191
Mgmt	20	/27	192.168.1.192	.193 – .222	.223
Link R1–R2	2	/30	192.168.1.224	.225 – .226	.227

6.13 Route summarization (in binary)

Combine contiguous networks into one route. Write the changing octet in binary and **count the bits that are the same in all of them**.

```
10.1.0.0  → 3rd octet 0 = 000000|00
10.1.1.0  → 3rd octet 1 = 000000|01
10.1.2.0  → 3rd octet 2 = 000000|10
10.1.3.0  → 3rd octet 3 = 000000|11
Common bits: 8 + 8 + 6 = 22  →  summary 10.1.0.0/22
```

Smaller routing tables and more stability.

6.14 Practice questions (answers below)

1. Convert 150 to binary.
2. Convert 11110000 to decimal.
3. Convert hex 2F to decimal.
4. Network, broadcast and host range of 192.168.50.100/28?
5. How many usable hosts in a /22?
6. Smallest subnet for 50 hosts?

7. Are 10.0.0.70/26 and 10.0.0.130/26 in the same subnet?
8. How many /24 subnets fit in 172.16.0.0/16?
9. Summarize 192.168.16.0/24 to 192.168.23.0/24 into one route.
10. Last usable host of 10.10.10.0/23?

Answers:

#	Answer	How
1	10010110	128 + 16 + 4 + 2 = 150
2	240	128 + 64 + 32 + 16
3	47	2×16 + 15
4	Network .96, hosts .97 – .110, broadcast .111	/28 → block 16 → 96 ≤ 100 < 112
5	1,022	h = 10 → $2^{10} - 2$
6	/26 (62 hosts)	/27 gives only 30
7	No	/26 blocks: 64–127 and 128–191
8	256	Borrow 8 bits → 2^8
9	192.168.16.0/21	16 = 00010000, 23 = 00010111 → 5 common bits → 16 + 5 = 21
10	10.10.11.254	/23 → block 2 in 3rd octet → network 10.10.10.0, broadcast 10.10.11.255

6.15 More practice (answers below)

1. 172.20.77.200/19: network, broadcast, usable range?
2. 10.200.33.17/29: network and broadcast?
3. You have 10.50.0.0/16 and need 300 subnets: which prefix? How many hosts per subnet?
4. Which of these are valid host addresses in 192.168.4.0/22: 192.168.5.255, 192.168.7.255, 192.168.4.0?
5. Summarize 172.16.32.0/24 through 172.16.47.0/24.
6. VLSM from 10.0.0.0/24: 60 hosts, 28 hosts, 12 hosts, two /30 links. Allocate.

#	Answer	How
1	Network 172.20.64.0, broadcast 172.20.95.255, hosts .64.1 – .95.254	/19 → 3rd octet mask 224 → block 32 → 64 ≤ 77 < 96
2	Network 10.200.33.16, broadcast .23	/29 → block 8
3	/25 (2^9 = 512 ≥ 300 subnets), 126 hosts each	/16 + 9 borrowed bits
4	192.168.5.255 is valid; .7.255 is the broadcast; .4.0 is the network	/22 → 192.168.4.0 – 192.168.7.255
5	172.16.32.0/20	32 = 00100000, 47 = 00101111 → 4 common bits → 16 + 4
6	10.0.0.0/26, 10.0.0.64/27, 10.0.0.96/28, 10.0.0.112/30, 10.0.0.116/30	Largest first, each block starts at a multiple of its size

6.16 Verify IP settings on client operating systems

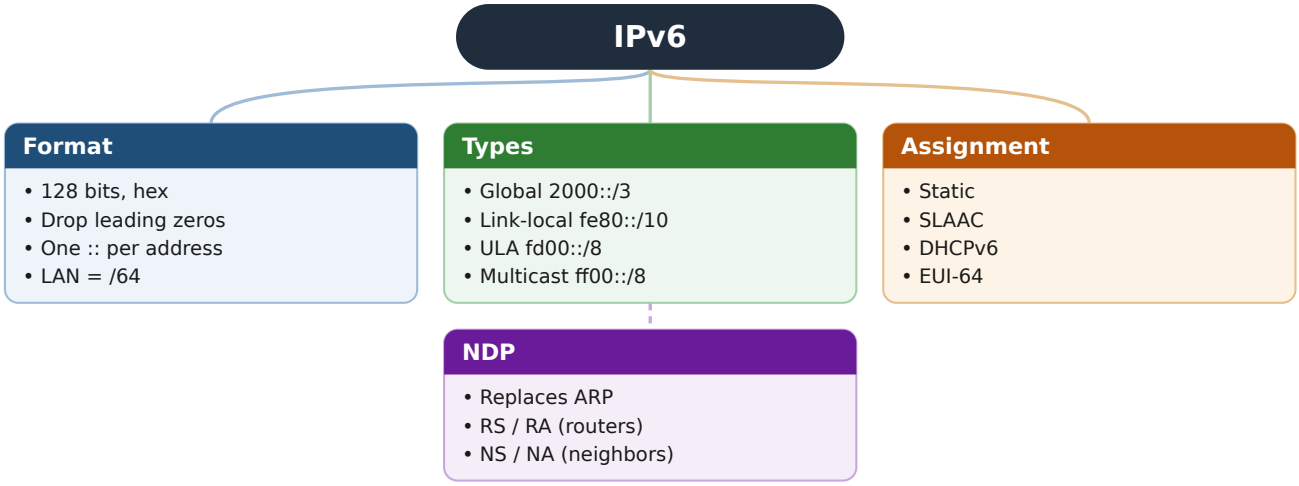
Task	Windows	macOS	Linux
IP, mask, gateway, DNS	`ipconfig /all`	`ifconfig`, `networksetup -getinfo Wi-Fi`, `scutil --dns`	`ip addr`, `ip route`, `resolvectl status`
Renew DHCP	`ipconfig /release` then `ipconfig /renew`	`sudo ipconfig set en0 DHCP`	`sudo dhclient -r && sudo dhclient` (or `nmcli con up <name>`)
Flush DNS cache	`ipconfig /flushdns`	`sudo dscacheutil -flushcache`	`resolvectl flush-caches`
Routing table	`route print`	`netstat -rn`	`ip route`
ARP table	`arp -a`	`arp -a`	`ip neigh`

What to verify, in order: an IP address in the right subnet (not 169.254.x.x = APIPA, meaning DHCP failed), the correct mask, a default gateway **in the same subnet**, working DNS servers. Then test: ping the gateway → ping a remote IP → resolve a name.

REMEMBER

- Mask octets you will see: **128, 192, 224, 240, 248, 252, 254, 255** (block sizes 128, 64, 32, 16, 8, 4, 2, 1).
- **Network** = host bits all 0 · **Broadcast** = host bits all 1 · usable hosts in between.
- Quick hosts → prefix: 14 → /28 · 30 → /27 · 62 → /26 · 126 → /25 · 254 → /24 · 510 → /23.
- Private ranges: **10/8, 172.16/12, 192.168/16**.

7. IPv6



7.1 Why IPv6

IPv4 has about 4.3 billion addresses: exhausted at the regional registries since the 2010s, kept alive by NAT. **IPv6** has 128-bit addresses (3.4 × 10^38), no need for NAT, simpler header, built-in autoconfiguration, and multicast instead of broadcast. Mobile networks, large ISPs and cloud providers run IPv6 widely; most networks are **dual stack** (IPv4 and IPv6 at once).

7.2 Format and shortening rules

- 128 bits = 8 groups ("hextets") of 4 hex digits separated by colons: `2001:0db8:0000:0000:0000:ff00:0042:8329`.
- **Rule 1:** remove leading zeros in any group: `0db8` → `db8`, `0042` → `42`, `0000` → `0`.
- **Rule 2:** replace **one** run of consecutive all-zero groups with `::` (only once per address, otherwise the length is ambiguous). If two runs are equal, shorten the first (RFC 5952 recommends the longest).

Full	Shortened
`2001:0db8:0000:0000:0000:ff00:0042:8329`	`2001:db8::ff00:42:8329`
`fe80:0000:0000:0000:0202:b3ff:fe1e:8329`	`fe80::202:b3ff:fe1e:8329`
`2001:0db8:0000:0001:0000:0000:0000:0001`	`2001:db8:0:1::1`
`0000:0000:0000:0000:0000:0000:0000:0001`	`::1`

Expand back by counting: `2001:db8::1` has 3 groups written, so `::` represents 5 groups of zeros.

7.3 Prefixes and subnetting

```
2001:0db8:acad : 0001 : 0000:0000:0000:0001
|<- global routing prefix (/48) ->|subnet|<----- interface ID (64 bits) ----->|
                                  | ID   |
```

Prefix	Typical owner
/32	ISP allocation from a registry
/48	One organization's site (65,536 /64 subnets)
/56	A home or small site (256 /64 subnets)
/64	**One LAN subnet** (required for SLAAC)
/127	Point-to-point router links (RFC 6164)
/128	One host / loopback

IPv6 subnetting is easy: with a /48, use the 4th hextet as the subnet ID: `2001:db8:acad:1::/64`, `2001:db8:acad:2::/64`, … `2001:db8:acad:ffff::/64`. Many teams encode meaning (VLAN 10 → `:10::/64`).

7.4 Address types

Type	Prefix	Use
Global unicast (GUA)	`2000::/3` (starts with 2 or 3)	Public, routable on the internet
Unique local (ULA)	`fc00::/7` (in practice `fd00::/8` + random 40-bit global ID)	Private, like RFC 1918; not routed on the internet
Link-local (LLA)	`fe80::/10`	Only valid on the local link; **every IPv6 interface has one automatically**; used by NDP, routing protocols, and as next hop
Loopback	`::1/128`	This host
Unspecified	`::/128`	"No address yet" (source during DAD)
Multicast	`ff00::/8`	One-to-many. `ff02::1` all nodes, `ff02::2` all routers, `ff02::5` / `ff02::6` OSPFv3, `ff02::9` RIPng, `ff02::1:2` DHCPv6 servers
Solicited-node multicast	`ff02::1:ff00:0/104` + last 24 bits of the unicast address	Efficient address resolution (replaces ARP broadcast)
Anycast	Any unicast address assigned to several devices	Routed to the nearest one (DNS, CDNs)
IPv4-mapped	`::ffff:0:0/96`	IPv4 addresses inside IPv6 sockets

There is **no broadcast** in IPv6.

7.5 Interface IDs: EUI-64 worked example

Modified EUI-64 builds the 64-bit interface ID from the 48-bit MAC:

```
MAC address:                    00 50 79 66 68 01
1. Split in half:               005079 | 666801
2. Insert FFFE in the middle:   0050 79FF FE66 6801
3. Flip the 7th bit (U/L bit) of the first byte:
   00 = 0000 0000  →  0000 0010 = 02
Interface ID:                   0250:79ff:fe66:6801
With prefix 2001:db8:acad:1::/64 → 2001:db8:acad:1:250:79ff:fe66:6801
```

Many operating systems now use **random/privacy interface IDs** instead (RFC 7217 stable-privacy, RFC 8981 temporary addresses), so the MAC isn't exposed.

7.6 Neighbor Discovery Protocol (NDP)

NDP (part of **ICMPv6**) replaces ARP and adds router discovery:

Message	ICMPv6 type	Purpose
Router Solicitation (RS)	133	Host asks "are there routers here?" (to `ff02::2`)
Router Advertisement (RA)	134	Router announces prefix, prefix length, default gateway (its link-local), flags for SLAAC/DHCPv6
Neighbor Solicitation (NS)	135	"What is the MAC of this IPv6 address?" (to the solicited-node multicast) and DAD
Neighbor Advertisement (NA)	136	Answer with the MAC
Redirect	137	Better next hop

Duplicate Address Detection (DAD): before using an address, a host sends an NS for it; if anyone answers, the address is a duplicate.

7.7 Address assignment methods

Method	How	Who provides DNS
Static	Configured manually	Manual
Static + EUI-64	Prefix manual, interface ID from MAC (`eui-64`)	Manual
SLAAC	Host takes the prefix from the RA and builds its own interface ID	RA (RDNSS option) or stateless DHCPv6
Stateless DHCPv6	SLAAC for the address + DHCPv6 for extra info (DNS, domain) (RA "O" flag)	DHCPv6
Stateful DHCPv6	DHCPv6 server assigns the address (RA "M" flag); the default gateway still comes from the RA	DHCPv6

7.8 Configuration and verification

```
R1(config)# ipv6 unicast-routing                          ! enable IPv6 routing (sends RAs)
R1(config)# interface g0/0
R1(config-if)# ipv6 address 2001:db8:acad:1::1/64         ! static GUA
R1(config-if)# ipv6 address fe80::1 link-local            ! easy-to-read link-local
R1(config-if)# no shutdown
R1(config)# interface g0/1
R1(config-if)# ipv6 address 2001:db8:acad:12::/64 eui-64  ! interface ID from the MAC
R1(config)# interface g0/2
R1(config-if)# ipv6 address autoconfig                    ! act as a SLAAC client
R1(config-if)# ipv6 enable                                ! link-local only, no global address

R1# show ipv6 interface brief
R1# show ipv6 interface g0/0           ! addresses, joined multicast groups, ND settings
R1# show ipv6 neighbors                ! like the ARP table
R1# show ipv6 route
R1# ping 2001:db8:acad:2::10
```

IPv6 static routes

```
R1(config)# ipv6 route 2001:db8:acad:2::/64 2001:db8:acad:12::2          ! next-hop GUA
R1(config)# ipv6 route 2001:db8:acad:2::/64 g0/1 fe80::2                 ! link-local next hop needs
                                                                          ! the exit interface
R1(config)# ipv6 route ::/0 2001:db8:acad:12::2                           ! default route
R1(config)# ipv6 route 2001:db8:acad:2::10/128 2001:db8:acad:12::2       ! host route
R1(config)# ipv6 route ::/0 2001:db8:ffff::1 200  ! floating default (AD 200)
```

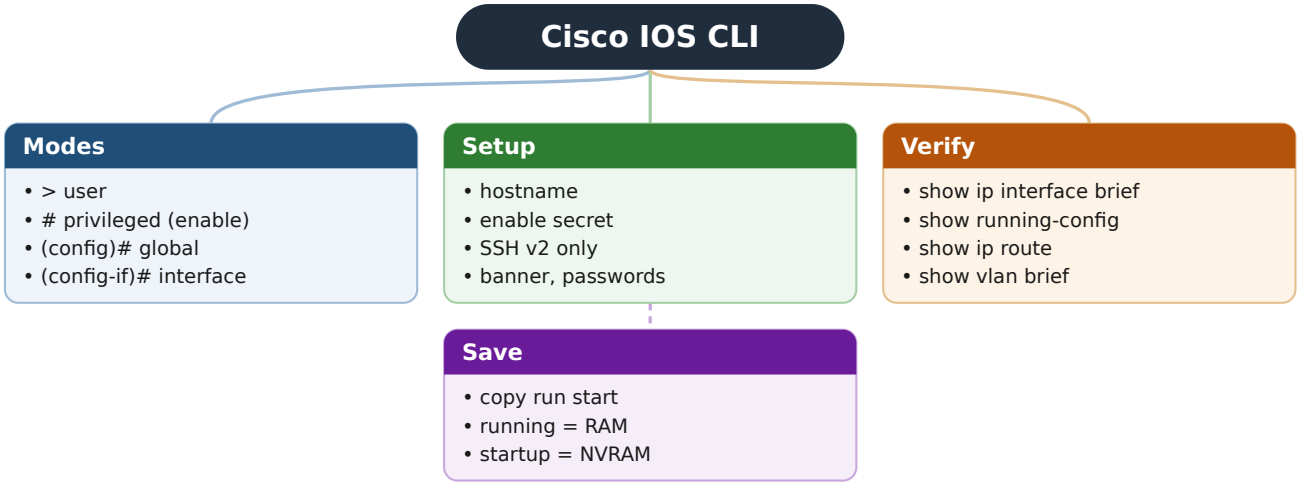
Transition mechanisms (awareness)

Dual stack (both protocols on every device: preferred), **tunneling** (IPv6 inside IPv4: 6in4, GRE), **translation** (NAT64/DNS64 lets IPv6-only clients reach IPv4 servers).

REMEMBER

- IPv6 has **no broadcast**; it uses multicast. Every interface has a **link-local** address.

8. Cisco IOS CLI Essentials



8.1 Ways to access a device

Access	How	Notes
Console	Rollover/USB console cable, terminal program (PuTTY, SecureCRT), 9600 baud 8N1	Out-of-band; works without any network config: initial setup and recovery
AUX	Modem port (legacy)	Remote out-of-band
VTY (SSH)	Over the network to a management IP	Encrypted: the standard
VTY (Telnet)	Over the network	**Plain text**: never in production
HTTP / HTTPS GUI	Web interface (WLC, small switches)	HTTPS only
Cloud-managed	Dashboard in the vendor's cloud (Meraki)	Devices call home
APIs	NETCONF, RESTCONF, REST	Automation (chapter 18)

Out-of-band management (separate management network/console servers) keeps access during outages of the production network.

8.2 Device components and boot process

Memory	Holds	Survives reboot
ROM	Bootstrap, POST, ROMMON (recovery mode)	Yes
Flash	IOS image files	Yes
NVRAM	**startup-config**	Yes
RAM	Running IOS, **running-config**, routing/ARP/MAC tables, buffers	**No**

Boot sequence: **POST** (hardware test) → **bootstrap** loads → find and load **IOS** (from flash, as per `boot system` or the first image) → load **startup-config** from NVRAM into RAM as running-config (if none: setup dialog).

Configuration register (`show version` , last line): `0x2102` normal boot; `0x2142` ignore startup-config (used in **password recovery**: break into ROMMON, set 0x2142, boot, copy startup to running, change passwords, set 0x2102, save).

8.3 Command modes

```
Router>                          User EXEC: limited show commands
Router> enable
Router#                          Privileged EXEC: all show, debug, copy, reload
Router# configure terminal
Router(config)#                  Global configuration
Router(config)# interface g0/1
Router(config-if)#               Interface configuration
Router(config)# line vty 0 15
Router(config-line)#             Line configuration
Router(config)# router ospf 1
Router(config-router)#           Routing protocol configuration
Router(config-if)# exit          Up one level
Router(config-if)# end           Back to Privileged EXEC (or Ctrl+Z)
```

CLI help: `?` (list commands/arguments), `Tab` (complete), abbreviations (`sh ip int br`), `Ctrl+A/E` line start/end, up arrow (history), `do show ...` (show commands from config mode), `no <command>` (remove a setting), `default interface g0/1` (reset an interface).

Output filtering: `show running-config | include hostname` , `| section router ospf` , `| begin interface` , `| exclude unassigned` .

8.4 Basic device setup

```
Switch(config)# hostname SW1
SW1(config)# enable secret <strong-password>           ! hashed privileged password
SW1(config)# service password-encryption               ! obscure type 7 passwords in the config
SW1(config)# security passwords min-length 10
SW1(config)# banner motd #Authorized access only. Activity is logged.#
SW1(config)# no ip domain-lookup                       ! typos aren't treated as hostnames
SW1(config)# login block-for 120 attempts 3 within 60  ! slow down password guessing
SW1(config)# line console 0
SW1(config-line)# password <console-password>
SW1(config-line)# login
SW1(config-line)# logging synchronous                  ! log messages don't break your typing
SW1(config-line)# exec-timeout 10 0                    ! log out after 10 idle minutes
SW1(config)# interface vlan 1                          ! management IP on a switch (SVI)
SW1(config-if)# ip address 192.168.1.2 255.255.255.0
SW1(config-if)# no shutdown
SW1(config)# ip default-gateway 192.168.1.1            ! L2 switch needs this to be managed remotely
SW1# copy running-config startup-config                ! SAVE (or: write memory)
```

Password type	Example	Strength
Type 0	`password cisco`	Plain text
Type 7	`service password-encryption`	Weak, reversible: only hides from shoulder-surfing
Type 5	`enable secret` (MD5 legacy)	Hashed
Type 8 / 9	`enable algorithm-type scrypt secret ...`	Strong (PBKDF2 / scrypt): preferred

`enable password` is weak; always use `enable secret` (if both exist, secret wins).

Config	Stored in	Meaning
running-config	RAM	Active now; lost on reboot unless saved
startup-config	NVRAM	Loaded at boot

`copy running-config startup-config` saves · `copy startup-config running-config` **merges** (doesn't replace) · `erase startup-config` + `reload` = factory reset (on switches also `delete vlan.dat`).

8.5 SSH instead of Telnet

```
SW1(config)# ip domain-name lab.local                 ! needed to generate the RSA key
SW1(config)# crypto key generate rsa modulus 2048
SW1(config)# ip ssh version 2
SW1(config)# username admin privilege 15 algorithm-type scrypt secret <strong-password>
SW1(config)# ip access-list standard MGMT-HOSTS
SW1(config-std-nacl)# permit 10.10.10.0 0.0.0.255
SW1(config)# line vty 0 15
SW1(config-line)# login local                         ! use the local user database (or AAA)
SW1(config-line)# transport input ssh                 ! no Telnet
SW1(config-line)# access-class MGMT-HOSTS in          ! only the admin subnet may connect
SW1# show ip ssh
SW1# show ssh
```

8.6 File management and backups

```
R1# show flash:                                 ! IOS images and files
R1# show version                                ! IOS version, uptime, image file, config register
R1# copy running-config tftp:                   ! backup to a TFTP server (prompts for address)
R1# copy running-config scp://admin@10.0.0.50/r1.cfg      ! secure backup
R1# copy tftp: flash:                           ! upload a new image
R1(config)# boot system flash:c2900-universalk9-mz.SPA.157-3.M8.bin
R1# verify /md5 flash:<image>                   ! check image integrity
R1# reload
```

Keep configuration backups versioned (Git via automation, or tools like Oxidized/RANCID), and verify image checksums against the vendor's published values before upgrading.

8.7 Most-used show commands

Command	Shows
`show running-config`	Current configuration
`show ip interface brief`	Interfaces, IPs, status/protocol (first command to run)
`show interfaces g0/1`	Detailed counters, errors, speed/duplex, MTU
`show interfaces status`	Switch ports: VLAN, speed, duplex, status
`show ip route`	Routing table
`show vlan brief`	VLANs and their access ports
`show interfaces trunk`	Trunk ports, native VLAN, allowed/active VLANs
`show mac address-table`	Learned MACs

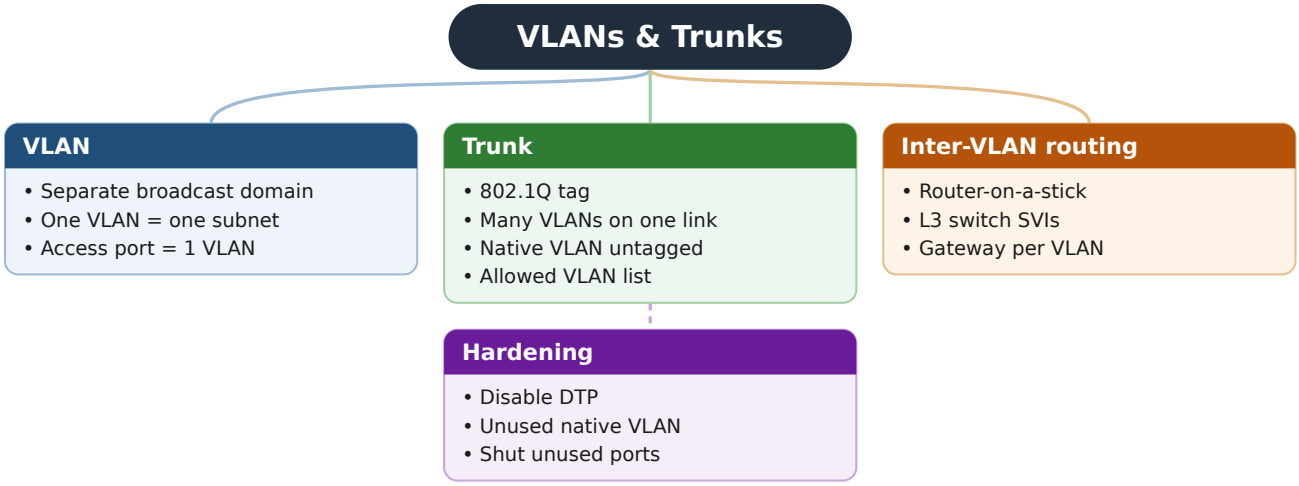
Command	Shows
`show spanning-tree`	STP root, port roles/states
`show etherchannel summary`	Port-channels and members
`show cdp neighbors detail` / `show lldp neighbors`	Directly connected devices
`show ip ospf neighbor`	OSPF neighbors
`show ip nat translations`	NAT entries
`show access-lists`	ACLs and hit counters
`show ip dhcp binding`	DHCP leases
`show version`	IOS version, uptime, hardware, config register
`show logging`	Log messages
`show processes cpu sorted`	CPU usage
`show tech-support`	Everything (for vendor support cases)

Debugging: `debug ip ospf adj`, `debug ip dhcp server events`; always `undebug all` afterwards and avoid debug on busy production devices (CPU impact). Use `terminal monitor` to see logs/debug output over SSH.

REMEMBER

- Unsaved changes are lost on reboot: **copy running-config startup-config**.
- First command when troubleshooting a device: **show ip interface brief**.

9. VLANs & Trunking



9.1 VLANs

- **What:** a **V**irtual **LAN** splits one physical switch (or many switches) into separate logical networks: separate **broadcast domains**.
- **Why:** security (separate HR, guests, servers, cameras), smaller broadcast domains, grouping by function instead of physical location, separate QoS (voice).
- **Rule:** one VLAN = one IP subnet. Devices in different VLANs need a **router or Layer 3 switch** to communicate.

VLAN range	Use
1	Default VLAN: all ports start here; can't be deleted. Don't use it for user or management traffic
2–1001	Normal range (stored in `vlan.dat` in flash)
1002–1005	Reserved for legacy FDDI/Token Ring; can't be deleted
1006–4094	Extended range
0, 4095	Reserved

```
SW1(config)# vlan 10
SW1(config-vlan)# name SALES
SW1(config)# vlan 20
SW1(config-vlan)# name HR
SW1(config)# vlan 100
SW1(config-vlan)# name VOICE
SW1(config)# vlan 999
SW1(config-vlan)# name NATIVE-UNUSED
SW1(config)# interface range f0/1 - 10
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 10
SW1(config-if-range)# switchport voice vlan 100        ! IP phone traffic tagged in VLAN 100
SW1(config-if-range)# spanning-tree portfast
SW1# show vlan brief
SW1# show interfaces f0/1 switchport                   ! mode, access VLAN, voice VLAN, trunking
```

Voice VLAN: one port connects an IP phone with a PC plugged into the phone. The phone tags its own traffic with the voice VLAN (and CoS 5 priority), the PC's frames stay untagged in the access (data) VLAN. The switch tells the phone the voice VLAN via **CDP** or **LLDP-MED**.

If an access port's VLAN doesn't exist on the switch, the port is **inactive** (no traffic): create the VLAN.

9.2 Trunks (IEEE 802.1Q)

- **Access port:** belongs to one data VLAN (plus optional voice VLAN); frames to/from end devices are **untagged**.
- **Trunk port:** carries many VLANs between switches, to routers, to servers/hypervisors and to APs. Each frame gets a **4-byte 802.1Q tag** inserted after the source MAC.

802.1Q tag field	Bits	Meaning
TPID	16	`0x8100`: "this frame is tagged"
PCP (CoS)	3	Priority 0–7 (voice = 5)
DEI	1	Drop eligible
VID	12	VLAN ID 0–4095 (usable 1–4094)

- **Native VLAN:** frames in this VLAN cross the trunk **untagged** (default VLAN 1). It must **match on both ends** (CDP reports a "native VLAN mismatch"; traffic leaks between VLANs). Best practice: set the native VLAN to an **unused** VLAN (e.g. 999) on all trunks, or tag it (`vlan dot1q tag native`).
- **Allowed VLAN list:** limit trunks to the VLANs that are needed (prunes broadcasts and limits exposure).

```
SW1(config)# interface g0/1
SW1(config-if)# switchport trunk encapsulation dot1q    ! only on switches that also support ISL
SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport trunk native vlan 999
SW1(config-if)# switchport trunk allowed vlan 10,20,100
SW1(config-if)# switchport trunk allowed vlan add 30  ! without "add" the list is REPLACED
SW1(config-if)# switchport nonegotiate                  ! disable DTP
SW1# show interfaces trunk
Port   Mode  Encapsulation  Status    Native vlan
Gi0/1  on    802.1q         trunking  999
Port   Vlans allowed on trunk
Gi0/1  10,20,30,100
Port   Vlans allowed and active in management domain
Gi0/1  10,20,30,100
Port   Vlans in spanning tree forwarding state and not pruned
Gi0/1  10,20,30,100
```

DTP (Dynamic Trunking Protocol, Cisco)

Mode (switchport mode …)	Behavior
`access`	Always access
`trunk`	Always trunk (still sends DTP unless `nonegotiate`)
`dynamic desirable`	Actively tries to form a trunk
`dynamic auto`	Becomes trunk only if the other side asks (default on many switches)

Side A \ Side B	access	trunk	desirable	auto
access	access	mismatch	access	access
trunk	mismatch	trunk	trunk	trunk
desirable	access	trunk	trunk	trunk
auto	access	trunk	trunk	**access**

Security: DTP lets an attacker negotiate a trunk (VLAN hopping). Hard-code every port as `access` or `trunk` and add `switchport nonegotiate`.

VTP (VLAN Trunking Protocol, Cisco)

Propagates the VLAN database from a **server** to **clients** in the same VTP domain. A switch with a **higher revision number** added to the network can **overwrite and delete VLANs everywhere**. Common practice: `vtp mode transparent` (or `off`) and manage VLANs per switch or with automation.

9.3 Inter-VLAN routing

Method	How	Notes
Separate router interfaces	One router port per VLAN	Doesn't scale
Router-on-a-stick	One trunk to the router, a subinterface per VLAN	Small sites; one link carries all routed traffic
Layer 3 switch with SVIs	`ip routing` + `interface vlan X` per VLAN	Standard in campuses: fast, hardware routing

```
! Router-on-a-stick
R1(config)# interface g0/0
R1(config-if)# no shutdown
R1(config)# interface g0/0.10
R1(config-subif)# encapsulation dot1Q 10
R1(config-subif)# ip address 192.168.10.1 255.255.255.0
R1(config)# interface g0/0.20
R1(config-subif)# encapsulation dot1Q 20
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
R1(config)# interface g0/0.999
R1(config-subif)# encapsulation dot1Q 999 native         ! the native VLAN subinterface
```

```
! Layer 3 switch
SW1(config)# ip routing
SW1(config)# interface vlan 10
SW1(config-if)# ip address 192.168.10.1 255.255.255.0
SW1(config-if)# no shutdown
SW1(config)# interface vlan 20
SW1(config-if)# ip address 192.168.20.1 255.255.255.0
SW1(config-if)# no shutdown
SW1(config)# interface g0/24                             ! routed port toward the core/router
SW1(config-if)# no switchport
SW1(config-if)# ip address 10.0.0.2 255.255.255.252
```

An **SVI is up/up** only if the VLAN exists and at least one port in that VLAN (access or trunk carrying it) is up and forwarding. Hosts in VLAN 10 use `192.168.10.1` as their default gateway.

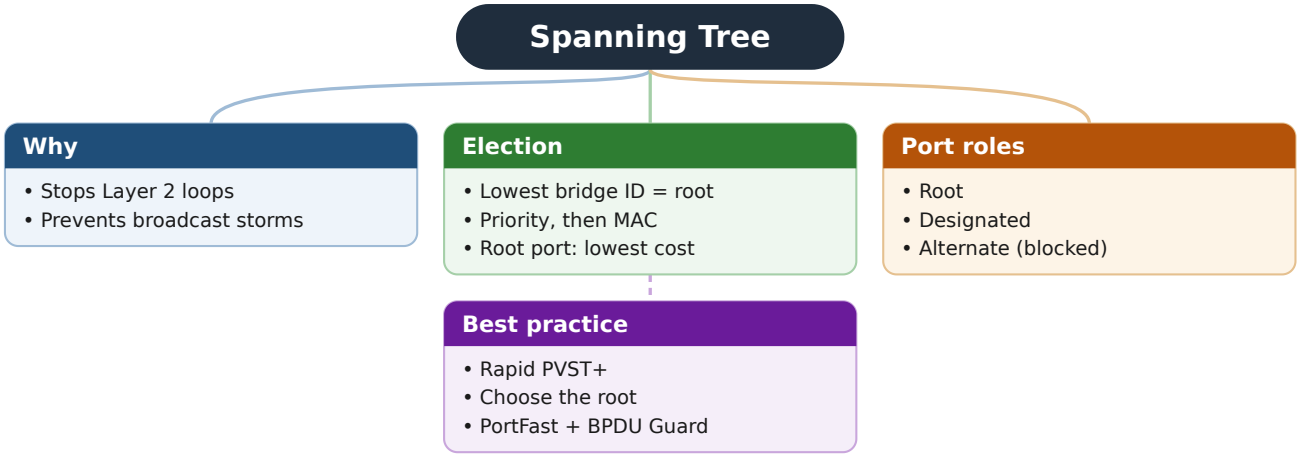
9.4 VLAN troubleshooting

Symptom	Check
Host can't reach others in the same VLAN	Port in the right VLAN (`show vlan brief`), VLAN exists, port not err-disabled, IP in the right subnet
Same VLAN works across one switch but not across the trunk	`show interfaces trunk`: trunk up? VLAN allowed? VLAN created on both switches? Native VLAN match?
VLANs can't reach each other	Gateway on hosts, SVI/subinterface up with the right IP, `ip routing` on the L3 switch, encapsulation VLAN ID correct
Access port shows "inactive"	VLAN doesn't exist on that switch

REMEMBER

- Different VLANs **cannot talk without a router or L3 switch**.
- Trunk problems: VLAN not allowed, native VLAN mismatch, trunk not up.

10. Spanning Tree Protocol (STP)



10.1 Why STP exists

Redundant links between switches create **Layer 2 loops**. Ethernet frames have **no TTL**, so a broadcast circulates forever: **broadcast storm** (links saturated), **MAC table instability** (MACs flap between ports), **duplicate frames**. A loop can take down a whole network in seconds. STP (IEEE 802.1D) logically **blocks** redundant ports so there is exactly one active path, and re-opens them when the active path fails.

10.2 Bridge ID and BPDUs

Switches exchange **BPDUs** (Bridge Protocol Data Units) every 2 seconds (hello time).

```
Bridge ID (8 bytes) = Bridge priority (4 bits) + Extended system ID = VLAN (12 bits) + MAC (48 bits)
Default priority 32768 → in VLAN 10 the switch shows priority 32778 (32768 + 10)
Priority is set in steps of 4096: 0, 4096, 8192, ... 61440
```

10.3 The election: four steps

1. **Elect one root bridge** (per VLAN in PVST+): the **lowest bridge ID** (lowest priority; on a tie, lowest MAC). All of the root's ports are **designated** (forwarding).
2. **Each non-root switch picks one root port**: its port with the lowest **root path cost** (sum of port costs toward the root). Tie-breakers in order: lowest **neighbor bridge ID** → lowest **neighbor port priority** (default 128) → lowest **neighbor port number**.
3. **Each segment (link) gets one designated port**: on the switch with the lowest root path cost for that segment (tie: lowest bridge ID). Designated ports forward.
4. **All remaining ports block** (802.1D blocking / RSTP alternate or backup).

Link speed	Short cost (802.1D-1998)	Long cost (802.1D-2004)
10 Mbps	100	2,000,000
100 Mbps	19	200,000
1 Gbps	4	20,000
10 Gbps	2	2,000

Link speed	Short cost (802.1D-1998)	Long cost (802.1D-2004)
100 Gbps	—	200

Worked example

```
              SW1 (priority 32768, MAC 0000.0000.1111)  ← lowest MAC: ROOT
            Gi0/1 (D)            Gi0/2 (D)
              | 1G                   | 1G
      Gi0/1 (Root)                 Gi0/1 (Root)
SW2 (32768, ...2222) ── 1G ── SW3 (32768, ...3333)
           Gi0/2 (D)        Gi0/2 (Alternate: BLOCKED)
```

- Root: SW1 (equal priority, lowest MAC).
- SW2 and SW3: root port = Gi0/1 (cost 4 directly to the root vs 8 via the other switch).
- SW2–SW3 link: both have root path cost 4; tie → lower bridge ID = **SW2** → SW2 Gi0/2 is designated, **SW3 Gi0/2 blocks**.

Never leave root election to chance (the oldest switch with the lowest MAC would win): set the core/distribution switch as root.

10.4 Port states, roles and timers

802.1D state	Duration	Learns MACs	Forwards
Blocking	Up to max age (20 s)	No	No (receives BPDUs)
Listening	Forward delay (15 s)	No	No
Learning	Forward delay (15 s)	Yes	No
Forwarding	—	Yes	Yes
Disabled	—	—	Shut down

Classic STP convergence: 30–50 seconds. Timers: **hello 2 s, forward delay 15 s, max age 20 s** (set on the root).

10.5 STP versions

Version	Standard	Note
STP	802.1D	One tree for all VLANs, slow
PVST+	Cisco	One 802.1D tree **per VLAN** (load-share: different roots for different VLANs)
RSTP	802.1w	Rapid convergence (about 1–2 s) using proposal/agreement handshakes
Rapid PVST+	Cisco	RSTP per VLAN: the default/recommended on Cisco switches and the CCNA focus
MSTP	802.1s	Maps many VLANs to a few instances (large networks)

RSTP roles: root, designated, **alternate** (backup path to the root, discarding), **backup** (backup designated port on a shared segment), disabled. **RSTP states: discarding** (replaces blocking/listening), learning, forwarding. **RSTP link types: point-to-point** (full duplex, fast handshake), **shared** (half duplex), **edge** (connects to an end device = PortFast).

10.6 Configuration and protection features

```
SW1(config)# spanning-tree mode rapid-pvst
SW1(config)# spanning-tree vlan 10,20 root primary  ! priority 24576 (or below the current root)
SW2(config)# spanning-tree vlan 10,20 root secondary    ! 28672: backup root
SW1(config)# spanning-tree vlan 30 priority 4096        ! or set explicitly (multiple of 4096)

SW3(config)# interface range f0/1 - 24                  ! access ports to end devices
SW3(config-if-range)# spanning-tree portfast  ! skip listening/learning: forward immediately
SW3(config-if-range)# spanning-tree bpduguard enable  ! err-disable if a BPDU arrives (rogue switch)
SW3(config)# spanning-tree portfast default              ! or globally for all access ports
SW3(config)# spanning-tree portfast bpduguard default

SW1(config)# interface g0/3  ! port toward a switch that must never be root
SW1(config-if)# spanning-tree guard root
SW1(config)# interface g0/1                              ! uplinks with blocked ports
SW1(config-if)# spanning-tree guard loop

SW1# show spanning-tree vlan 10
SW1# show spanning-tree summary
SW1# show spanning-tree interface g0/1 detail
```

Feature	Protects against	Action
PortFast	Slow host connections (30 s wait; DHCP timeouts)	Port goes straight to forwarding. **Only on ports to end devices** (or trunk to a server with `portfast trunk`)
BPDU Guard	Someone plugging a switch into a PortFast port	Port is **err-disabled** when a BPDU is received
BPDU Filter	Sending/receiving BPDUs on a port	Stops BPDUs (dangerous: can create loops; use rarely)
Root Guard	A downstream switch becoming root (superior BPDUs)	Port goes **root-inconsistent** (blocking) while superior BPDUs arrive
Loop Guard	A blocked port wrongly moving to forwarding because BPDUs stopped arriving (unidirectional link)	Port goes **loop-inconsistent**

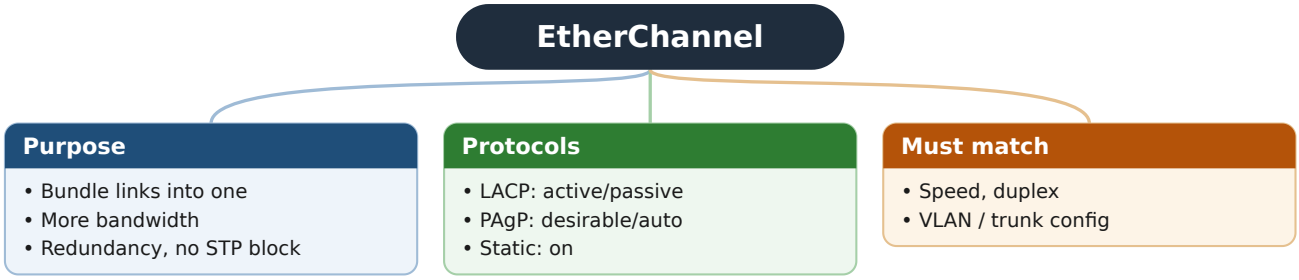
Recover err-disabled ports: remove the cause, then `shutdown` / `no shutdown`, or configure `errdisable recovery cause bpduguard` with an interval.

```
SW1# show spanning-tree vlan 10
VLAN0010
  Spanning tree enabled protocol rstp
  Root ID    Priority    24586
             Address     0000.0000.1111
             This bridge is the root
  Bridge ID  Priority    24586  (priority 24576 sys-id-ext 10)
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi0/1               Desg FWD 4         128.25   P2p
Gi0/2               Desg FWD 4         128.26   P2p
```

REMEMBER

- Root bridge = **lowest priority, then lowest MAC**. Set it deliberately on a core switch.
- Costs: 100 Mbps = 19 · 1 Gbps = 4 · 10 Gbps = 2.

11. EtherChannel (Link Aggregation)



11.1 What and why

EtherChannel bundles 2–8 physical links into **one logical link** (port-channel):

- **More bandwidth** (traffic shared across members).
- **Redundancy:** a member fails, traffic moves to the others without STP reconverging.
- **STP sees one link**, so no member is blocked.

Load is shared **per flow** (hash of MAC/IP/ports), not per packet: a single flow uses one member link. Choose the hash for your traffic: `port-channel load-balance src-dst-ip` (global).

11.2 Negotiation protocols

Protocol	Standard	Modes	Forms a channel when
LACP	IEEE 802.3ad / 802.1AX	`active` (initiates), `passive` (responds)	active–active, active–passive (**passive–passive fails**)
PAgP	Cisco	`desirable` (initiates), `auto` (responds)	desirable–desirable, desirable–auto (auto–auto fails)
Static	—	`on`	on–on only (no negotiation: misconfigurations cause loops/black holes)

Use **LACP active** on both sides; it works with servers, hypervisors, firewalls and other vendors. LACP supports up to 16 members (8 active + 8 hot-standby).

11.3 Configuration

```
! Layer 2 EtherChannel trunk
SW1(config)# interface range g0/1 - 2
SW1(config-if-range)# channel-group 1 mode active        ! LACP; creates interface port-channel 1
SW1(config)# interface port-channel 1
SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport trunk native vlan 999
SW1(config-if)# switchport trunk allowed vlan 10,20,100

! Layer 3 EtherChannel (routed link between L3 switches)
SW1(config)# interface range g0/3 - 4
SW1(config-if-range)# no switchport
SW1(config-if-range)# channel-group 2 mode active
SW1(config)# interface port-channel 2
SW1(config-if)# no switchport
SW1(config-if)# ip address 10.0.12.1 255.255.255.252

SW1# show etherchannel summary
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        S - Layer2      R - Layer3      U - in use
Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------
1      Po1(SU)         LACP      Gi0/1(P)    Gi0/2(P)
2      Po2(RU)         LACP      Gi0/3(P)    Gi0/4(P)
SW1# show etherchannel port-channel
SW1# show lacp neighbor
```

11.4 Requirements and troubleshooting

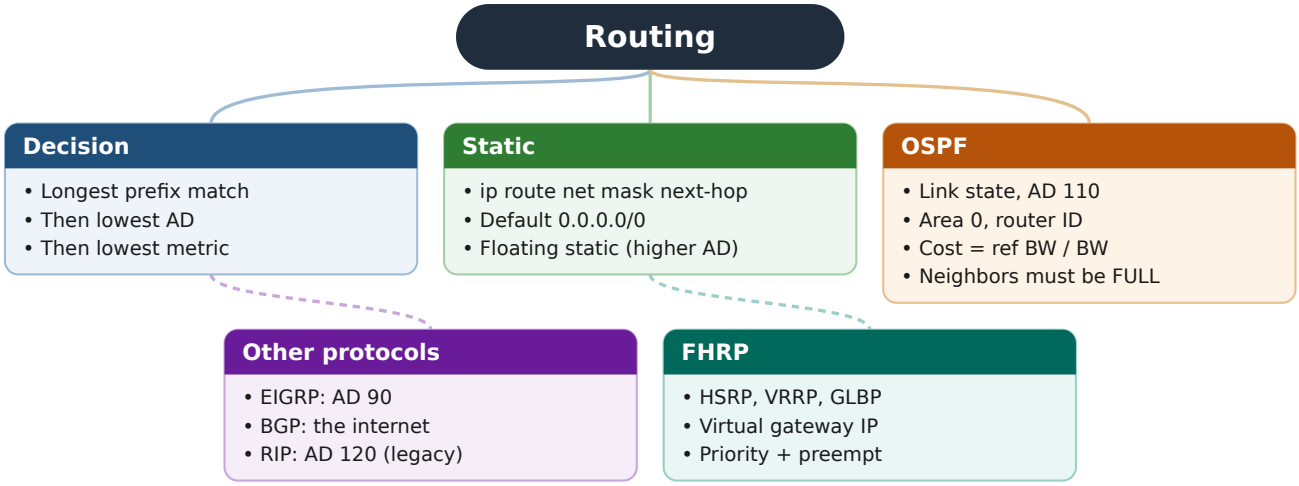
All members must match: **speed, duplex, switchport mode (access/trunk), access VLAN / allowed VLANs / native VLAN**, and Layer 2 vs Layer 3. Configure settings on the **port-channel interface** (they're inherited by the members).

Symptom	Cause
Members show `(I)` stand-alone or `(s)` suspended	Mismatched settings, or the other side isn't configured for the channel
Channel doesn't form	passive–passive or auto–auto modes; LACP vs PAgP mismatch
err-disabled ports	`on` mode on one side only → EtherChannel misconfig guard
Uneven load	Hash method doesn't fit traffic (e.g. all traffic between two IPs)

REMEMBER

- Use **LACP active** on both sides (open standard).

12. Routing



12.1 How a router forwards a packet

1. Receives a frame addressed to its MAC, checks the FCS, removes the Layer 2 header.
2. Reads the **destination IP** and looks it up in the **routing table**: chooses the **longest prefix match** (most specific route).
3. Decrements **TTL** (drops the packet at 0 and sends ICMP time exceeded), recomputes the IPv4 checksum.
4. Finds the next hop's MAC (ARP cache / ARP request), builds a **new frame** and sends it out the exit interface.
5. No matching route and no default route → **drop** and send ICMP "destination unreachable".

Choosing between routes: three rules in order

Order	Rule	Compares
1	**Longest prefix match**	A /24 beats a /16 beats a /0, regardless of source
2	**Administrative distance**	For the *same prefix* from different sources: lower AD wins (installed in the table)
3	**Metric**	For the same prefix from the same protocol: lower metric wins (equal metrics → ECMP load sharing)

Example: routes `10.0.0.0/8` (static), `10.1.0.0/16` (OSPF), `10.1.1.0/24` (EIGRP): a packet to `10.1.1.5` uses the **/24**; a packet to `10.1.2.5` uses the **/16**; a packet to `10.200.0.1` uses the **/8**.

12.2 Reading the routing table

```
R1# show ip route
Codes: L - local, C - connected, S - static, O - OSPF, D - EIGRP, B - BGP, * - candidate default
Gateway of last resort is 203.0.113.1 to network 0.0.0.0

S*    0.0.0.0/0 [1/0] via 203.0.113.1
      10.0.0.0/8 is variably subnetted, 4 subnets, 3 masks
C        10.1.1.0/24 is directly connected, GigabitEthernet0/0
L        10.1.1.1/32 is directly connected, GigabitEthernet0/0
C        10.1.12.0/30 is directly connected, GigabitEthernet0/1
O        10.2.2.0/24 [110/2] via 10.1.12.2, 00:05:12, GigabitEthernet0/1
S     192.168.50.0/24 [1/0] via 10.1.12.2
```

Part	Example	Meaning
Route source code	`O`	Learned via OSPF
Destination prefix / mask	`10.2.2.0/24`	Network the route reaches
[AD/metric]	`[110/2]`	Administrative distance 110, OSPF cost 2
Next hop	`via 10.1.12.2`	The neighbor router to send to
Age	`00:05:12`	Time since learned
Exit interface	`GigabitEthernet0/1`	Where packets leave
Gateway of last resort	`0.0.0.0/0`	Default route used when nothing more specific matches

Connected (C) routes appear automatically for each up/up interface with an IP; **local (L)** /32 routes are the router's own addresses.

12.3 Administrative distance

Source	AD
Connected	0
Static	1
eBGP	20
EIGRP (internal)	90
OSPF	110
IS-IS	115
RIP	120
EIGRP (external)	170
iBGP	200
Unusable (never installed)	255

12.4 Static routing

Type	Example	Use
Network route	`ip route 192.168.50.0 255.255.255.0 10.1.12.2`	Reach one remote network
Default route	`ip route 0.0.0.0 0.0.0.0 203.0.113.1`	Everything else (internet edge, stub sites)
Host route	`ip route 10.9.9.9 255.255.255.255 10.1.12.2`	One specific host (/32)
Floating static	`ip route 0.0.0.0 0.0.0.0 198.51.100.1 5`	Backup with a higher AD: used only when the primary route disappears
Summary route	`ip route 172.16.0.0 255.255.248.0 10.1.12.2`	One route covering several networks

```
R1(config)# ip route 192.168.50.0 255.255.255.0 10.1.12.2          ! next-hop IP (recursive lookup)
R1(config)# ip route 192.168.50.0 255.255.255.0 g0/1               ! exit interface: p2p links only
R1(config)# ip route 192.168.50.0 255.255.255.0 g0/1 10.1.12.2     ! fully specified: best practice
R1(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.1                    ! primary default via ISP A
R1(config)# ip route 0.0.0.0 0.0.0.0 198.51.100.1 5                 ! floating default via ISP B
R1# show ip route static
R1# show ip route 192.168.50.10                                     ! which route a destination uses
```

Always think about the return path: R1 needs a route to the remote network, and the remote router needs a route back. One-way routing is the most common static routing mistake.

A static route is installed only if its next hop is reachable (or exit interface is up). A floating static backup only takes over when the primary disappears from the table: if the primary next hop is "up" but broken further away, the static route stays (use IP SLA tracking for that).

12.5 Dynamic routing protocols

Type	How	Examples
Distance vector	Learns routes from neighbors' tables ("routing by rumour"), periodic updates	RIP (metric = hop count, max 15)
Advanced distance vector	Distance vector with fast convergence and loop-free backups	EIGRP
Link state	Every router floods link information, builds a full map (LSDB) and runs **SPF (Dijkstra)**	**OSPF**, IS-IS
Path vector	Routes carry the list of autonomous systems they crossed; policy-based	BGP
IGP vs EGP	Inside one organization vs between organizations	OSPF, EIGRP, IS-IS vs BGP

Why dynamic routing: adapts automatically to failures, scales to many networks, less manual work.
Why static: simple, predictable, no overhead: good for stub networks and default routes.

12.6 OSPF (single-area OSPFv2: the CCNA focus)

Concepts

Term	Meaning
Router ID (RID)	32-bit unique ID. Chosen: manual `router-id` → highest **loopback** IP → highest active interface IP. Changes require `clear ip ospf process`
Area	Group of routers sharing the same LSDB. **Area 0** = backbone; multi-area designs connect all areas to area 0 through ABRs
Neighbor / adjacency	Routers that exchange Hellos (multicast **224.0.0.5**) / that fully synchronized their databases
LSA / LSDB	Link-State Advertisements (type 1 router LSA, type 2 network LSA from the DR, type 3 summary between areas) / the database (the map)
SPF	Dijkstra's algorithm computes the shortest-path tree from the LSDB
Cost	Metric = **reference bandwidth / interface bandwidth** (default reference 100 Mbps; minimum cost 1). Route cost = sum of outgoing interface costs
DR / BDR	On broadcast (Ethernet multi-access) segments a Designated Router and Backup DR reduce flooding; others (DROTHERs) form full adjacency only with the DR/BDR (multicast 224.0.0.6 to DR/BDR)
Timers	Hello 10 s, Dead 40 s on broadcast/point-to-point networks: must match

Cost table (default reference bandwidth 100 Mbps)

Interface	Cost (ref 100 Mbps)	Cost (ref 100 Gbps = 100000)
10 Mbps	10	10000
100 Mbps	1	1000
1 Gbps	**1** (same as 100M!)	100
10 Gbps	1	10

Set `auto-cost reference-bandwidth` to the same value on **all** routers so faster links get lower costs. Or set `ip ospf cost` per interface.

Packet types and neighbor states

OSPF packet	Purpose
1 Hello	Discover neighbors, keep alive, elect DR/BDR
2 DBD (Database Description)	Summary of the LSDB
3 LSR (Link-State Request)	Ask for missing LSAs
4 LSU (Link-State Update)	Send LSAs
5 LSAck	Acknowledge

States: Down → Init (got a Hello) → **2-Way** (saw itself in the neighbor's Hello; DROTHERs stay here with each other: normal) → ExStart (master/slave) → Exchange (DBDs) → Loading (LSR/LSU) → **Full** (synchronized).

DR/BDR election

- Highest **OSPF interface priority** wins (default 1; `ip ospf priority 0` = never DR/BDR).
- Tie: highest **router ID**.
- **Non-preemptive:** a better router joining later doesn't take over until the DR/BDR fails (or the process is cleared).
- Point-to-point links don't elect a DR: `ip ospf network point-to-point` on Ethernet router-to-router links speeds up adjacency and removes unnecessary DR/BDR.

Neighbor requirements (must match)

Same subnet and mask · same area ID · same hello and dead timers · same authentication · same MTU (otherwise stuck in ExStart/Exchange) · same network type (or compatible) · **unique** router IDs · interface not passive · stub area flags match. ACLs must not block protocol 89.

Configuration

```
R1(config)# interface loopback 0
R1(config-if)# ip address 1.1.1.1 255.255.255.255
R1(config)# router ospf 1                                  ! process ID: locally significant
R1(config-router)# router-id 1.1.1.1
R1(config-router)# auto-cost reference-bandwidth 100000    ! 100 Gbps reference, on ALL routers
R1(config-router)# network 10.1.1.0 0.0.0.255 area 0       ! interfaces in this range join area 0
R1(config-router)# network 10.1.12.0 0.0.0.3 area 0
R1(config-router)# network 1.1.1.1 0.0.0.0 area 0
R1(config-router)# passive-interface g0/0  ! advertise the LAN, but no Hellos to users
R1(config-router)# default-information originate           ! advertise this router's default route
! Alternative: enable per interface (clearer, preferred by many)
R1(config)# interface g0/1
R1(config-if)# ip ospf 1 area 0
R1(config-if)# ip ospf network point-to-point               ! router-to-router link, no DR election
R1(config-if)# ip ospf cost 10                              ! manual cost override
R1(config)# interface g0/2
R1(config-if)# ip ospf priority 255                         ! prefer as DR on this segment
```

Wildcard mask = 255.255.255.255 − subnet mask: /24 → `0.0.0.255`, /30 → `0.0.0.3`, single address → `0.0.0.0`. In the `network` command, 0 bits must match and 1 bits are "don't care".

Verification

Command	Shows
`show ip ospf neighbor`	Neighbor RID, priority, **state (FULL/DR, FULL/BDR, 2WAY/DROTHER)**, dead time, interface
`show ip ospf interface brief`	Interfaces in OSPF, area, cost, state (DR/BDR/DROTHER/P2P), neighbor count
`show ip ospf interface g0/1`	Network type, timers, priority, DR/BDR addresses
`show ip route ospf`	Routes learned by OSPF (`O` , `O*E2` for external default)
`show ip protocols`	Router ID, networks, passive interfaces, reference bandwidth, AD
`show ip ospf database`	The LSDB

```
R1# show ip ospf neighbor
Neighbor ID   Pri   State           Dead Time   Address       Interface
2.2.2.2         0   FULL/  -        00:00:35    10.1.12.2     GigabitEthernet0/1
3.3.3.3         1   FULL/DR         00:00:38    10.1.123.3    GigabitEthernet0/2
4.4.4.4         1   2WAY/DROTHER    00:00:31    10.1.123.4    GigabitEthernet0/2
```

OSPFv3 (IPv6) in brief

Same concepts, runs over IPv6 link-local addresses, enabled per interface:

```
R1(config)# ipv6 unicast-routing
R1(config)# ipv6 router ospf 1
R1(config-rtr)# router-id 1.1.1.1                          ! still a 32-bit ID
R1(config)# interface g0/1
R1(config-if)# ipv6 ospf 1 area 0
R1# show ipv6 ospf neighbor
```

12.7 EIGRP and BGP (awareness)

- **EIGRP:** Cisco advanced distance vector (published as an informational RFC), AD 90 internal / 170 external, composite metric from bandwidth and delay, **DUAL** algorithm with **successors** (best path) and **feasible successors** (pre-computed loop-free backups) for very fast convergence, supports unequal-cost load balancing.
- **BGP:** the routing protocol of the internet. Connects **autonomous systems (AS)**; runs over TCP 179 between configured peers; chooses paths by **attributes and policy** (weight, local preference, AS path length, MED…) rather than speed. eBGP between organizations (AD 20), iBGP inside one (AD 200). Used by ISPs, multihomed enterprises, data center fabrics, and cloud hybrid links (AWS Direct Connect, VPN with BGP).

12.8 First Hop Redundancy Protocols (FHRP)

Problem: hosts have one default gateway. If that router fails, the subnet loses connectivity, even though another router exists. **Solution:** two or more routers share a **virtual IP** (configured as the hosts' gateway) and a **virtual MAC**. One router actively forwards; another takes over on failure, sending gratuitous ARPs so switches learn the new location of the virtual MAC.

Protocol	Standard	Roles	Virtual MAC	Notes
HSRP	Cisco	Active / Standby (others listen)	v1: `0000.0c07.acXX` (XX = group), v2: `0000.0c9f.fXXX`	Default priority 100, highest wins (tie: highest IP); **preemption off by default**; hello 3 s, hold 10 s; v2 supports IPv6 and groups 0–4095
VRRP	IEEE/IETF (RFC 5798)	Master / Backup	`0000.5e00.01XX`	Preemption on by default; can use a real interface IP as the virtual IP
GLBP	Cisco	AVG + up to 4 AVFs	Several	Load-balances hosts across all gateways

```
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.10.2 255.255.255.0
R1(config-if)# standby version 2
R1(config-if)# standby 10 ip 192.168.10.1          ! virtual gateway IP for hosts
R1(config-if)# standby 10 priority 110             ! higher than R2 (100) → Active
R1(config-if)# standby 10 preempt                  ! take Active back after recovering
R1(config-if)# standby 10 track g0/1 20            ! lower priority by 20 if the uplink fails
R2(config)# interface g0/0
R2(config-if)# ip address 192.168.10.3 255.255.255.0
R2(config-if)# standby version 2
R2(config-if)# standby 10 ip 192.168.10.1
R2(config-if)# standby 10 preempt
R1# show standby brief
Interface   Grp  Pri P State   Active          Standby         Virtual IP
Gi0/0       10   110 P Active  local           192.168.10.3    192.168.10.1
```

Tip: align the HSRP active router with the STP root for that VLAN so traffic takes the direct path.

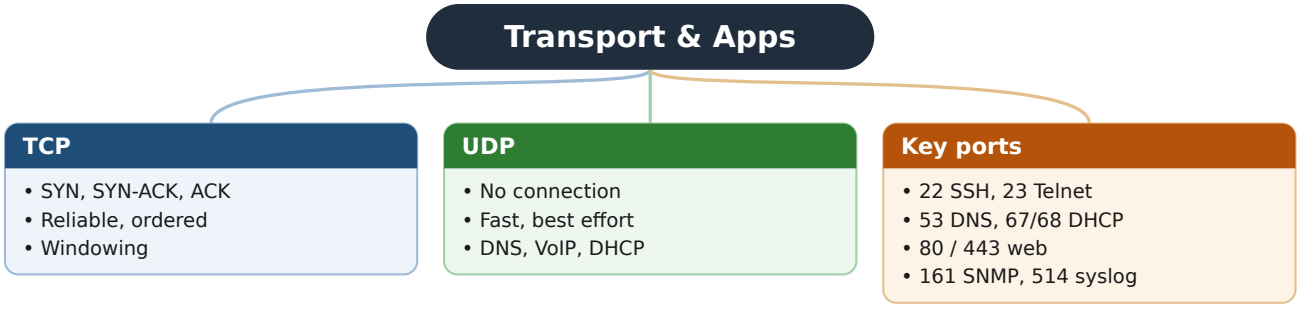
12.9 Routing troubleshooting

Symptom	Check
Can't reach a remote network	`show ip route <dest>` on each hop: is there a route? Is there a **return route**?
Route missing	Interface down (connected routes vanish), static next hop unreachable, OSPF neighbor not FULL, network not advertised (`network` statement/interface not in OSPF)
Wrong path used	Longest prefix (a more specific route elsewhere), AD (a static overriding OSPF), metrics/costs (reference bandwidth inconsistent)
OSPF neighbors stuck	INIT: one-way Hellos (ACL, layer 2 issue); 2-WAY: normal between DROTHERs; EXSTART/EXCHANGE: MTU mismatch; no neighbor at all: subnet/area/timers/auth/passive/duplicate RID
Intermittent reachability	Routing flaps (`show ip route` age keeps resetting), unstable link, duplicate IP
Traceroute loops between two routers	Routing loop: two routers point at each other (often static routes or a bad default)

REMEMBER

- AD: **Connected 0 · Static 1 · eBGP 20 · EIGRP 90 · OSPF 110 · RIP 120**.
- OSPF won't form? **subnet, area, timers, MTU, passive, router ID**.
- Always check the **return path**: routing must work in both directions.

13. Transport & Application Protocols



13.1 Ports and sockets

The transport layer delivers data to the right **application** on a host using **port numbers**. A connection is identified by the **5-tuple**: protocol, source IP, source port, destination IP, destination port. A **socket** = IP address + port.

Port ranges: **0–1023** well-known (servers), **1024–49151** registered, **49152–65535** dynamic/ephemeral (clients pick these as source ports).

```
Browser on 192.168.1.10 opens two tabs to the same server:
  192.168.1.10:51001 → 93.184.216.34:443   (tab 1)
  192.168.1.10:51002 → 93.184.216.34:443   (tab 2)   different source ports keep them separate
```

13.2 TCP in depth

Feature	How
Connection setup	Three-way handshake: SYN → SYN-ACK → ACK
Reliability	Every byte has a sequence number; the receiver acknowledges; unacknowledged data is **retransmitted** after a timeout or duplicate ACKs
Ordering	Receiver reorders segments by sequence number
Flow control	Receiver advertises a **window** (how many bytes it can buffer); the sender never exceeds it (**sliding window**)
Congestion control	Sender slows down when loss indicates congestion (slow start, congestion avoidance)
Segmentation	Data is split into segments no larger than the **MSS** (typically 1460 bytes = 1500 MTU − 40 bytes IP/TCP headers)
Termination	FIN → ACK, FIN → ACK (four-way), or **RST** to abort immediately

```
Client                                  Server (listening on 443)
  | -- SYN, seq=100 -------------------->  |
  | <-- SYN-ACK, seq=300, ack=101 -------  |
  | -- ACK, seq=101, ack=301 ----------->  |   connection ESTABLISHED
  | -- data, 500 bytes, seq=101 -------->  |
  | <-- ACK, ack=601 --------------------  |   "I expect byte 601 next"
  | -- FIN ---------------------------->  |
  | <-- ACK, then FIN ------------------  |
  | -- ACK ---------------------------->  |   closed (client waits in TIME_WAIT)
```

13.3 TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Acknowledgments, retransmission	None (application handles it if needed)
Order	Guaranteed	Not guaranteed
Flow / congestion control	Yes	No
Header	20–60 bytes	8 bytes
Speed / overhead	More overhead, adds latency on loss	Minimal overhead, low latency
Use	Web (HTTP/1.1, HTTP/2), SSH, email, file transfer, databases	DNS, DHCP, VoIP/video (RTP), SNMP, NTP, syslog, TFTP, QUIC (HTTP/3)

Real-time voice/video prefers UDP: a late retransmitted packet is useless, it's better to skip it.

13.4 Common ports and protocols

Protocol	Port	Transport	Purpose
FTP	20 (data), 21 (control)	TCP	File transfer (plain text)
SSH / SFTP / SCP	22	TCP	Secure remote login / file transfer
Telnet	23	TCP	Remote login, plain text: avoid
SMTP	25 (587 submission)	TCP	Sending email
DNS	53	UDP (TCP for large responses/zone transfers)	Name resolution
DHCP	67 server / 68 client	UDP	Address assignment
TFTP	69	UDP	Simple file transfer (device images, configs, IP phones)
HTTP / HTTPS	80 / 443	TCP (HTTP/3: UDP 443 QUIC)	Web, REST APIs
POP3 / IMAP	110 / 143 (995 / 993 secure)	TCP	Reading email
NTP	123	UDP	Time sync
SNMP	161 (traps 162)	UDP	Device monitoring

Protocol	Port	Transport	Purpose
LDAP / LDAPS	389 / 636	TCP	Directory
Syslog	514	UDP	Log messages
BGP	179	TCP	Internet routing
RADIUS	1812 auth / 1813 accounting	UDP	Network access AAA
TACACS+	49	TCP	Device administration AAA
NETCONF	830	TCP (SSH)	Network automation
RDP	3389	TCP	Windows remote desktop

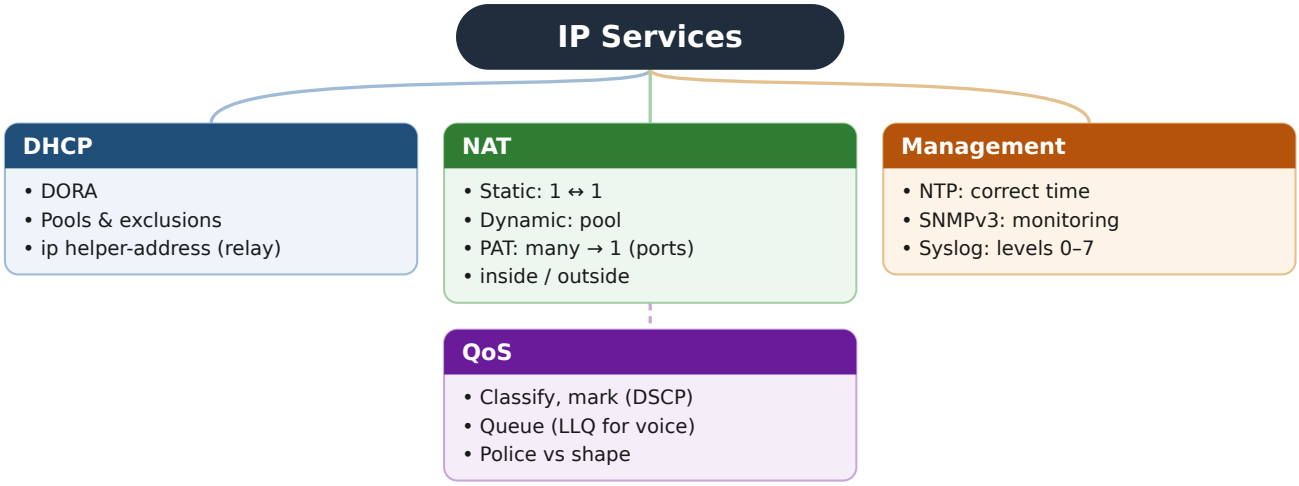
13.5 Key application protocols

- **HTTP:** request (method `GET/POST/PUT/PATCH/DELETE`, path, headers, body) and response (status code, headers, body). HTTPS = HTTP over **TLS** (certificates, encryption, integrity). HTTP/2 multiplexes requests over one connection; HTTP/3 runs over QUIC (UDP).
- **DNS:** see 14.2.
- **Email:** SMTP sends between servers; IMAP/POP3 read mailboxes.

REMEMBER

- Ephemeral (client) ports: **49152–65535**. Well-known: **0–1023**.

14. IP Services



14.1 DHCP (Dynamic Host Configuration Protocol)

Gives hosts their IP address, mask, default gateway, DNS servers, domain name and lease time automatically.

```
Client (no IP, UDP 68)                          DHCP server (UDP 67)
  | -- DISCOVER (broadcast 255.255.255.255) -->|  "Any DHCP servers?"
  | <-- OFFER (proposed IP, options) ----------|
  | -- REQUEST (broadcast: "I take this one") ->|  others see which offer was accepted
  | <-- ACK (confirmed, lease starts) ---------|
```

DORA = Discover, Offer, Request, Acknowledge. The client renews at **50%** of the lease (T1, unicast to the server) and rebinds at **87.5%** (T2, broadcast to any server). `DHCPRELEASE` returns the address; `DHCPDECLINE` reports a conflict.

Router as DHCP server

```
R1(config)# ip dhcp excluded-address 192.168.10.1 192.168.10.20  ! gateways, servers, printers
R1(config)# ip dhcp pool SALES
R1(dhcp-config)# network 192.168.10.0 255.255.255.0
R1(dhcp-config)# default-router 192.168.10.1
R1(dhcp-config)# dns-server 192.168.1.53 8.8.8.8
R1(dhcp-config)# domain-name lab.local
R1(dhcp-config)# lease 7                                          ! days
R1# show ip dhcp binding
R1# show ip dhcp pool
R1# show ip dhcp conflict
```

DHCP relay

DHCP Discover is a **broadcast**, and routers don't forward broadcasts. With a central DHCP server in another subnet, configure a **relay agent** on the gateway interface of the client subnet. It converts the broadcast to a unicast to the server and fills the `giaddr` field so the server knows which pool to use.

```
R1(config)# interface g0/0.10                     ! gateway of the client VLAN
R1(config-subif)# ip helper-address 10.0.0.50     ! the DHCP server
```

Router as DHCP client (e.g. toward an ISP)

```
R1(config)# interface g0/1
R1(config-if)# ip address dhcp
R1# show dhcp lease
```

Client commands: `ipconfig /release` + `ipconfig /renew` (Windows), `dhclient` / `nmcli` (Linux). A client with **169.254.x.x** (APIPA) never got an answer.

14.2 DNS (Domain Name System)

Translates names to IP addresses (and more). Hierarchy: **root** (`.`) → **TLD** (`.com` , `.in`) → **authoritative** servers for the domain.

```
PC → recursive resolver (ISP / 8.8.8.8 / company DNS): "www.example.com?"
     resolver → root server:         "ask the .com servers"
     resolver → .com TLD server:     "ask ns1.example.com"
     resolver → authoritative server:"www.example.com = 93.184.216.34, TTL 3600"
     resolver caches the answer for the TTL and replies to the PC
```

Record	Purpose
A / AAAA	Name → IPv4 / IPv6
CNAME	Alias to another name
MX	Mail servers for the domain
TXT	Text: SPF, DKIM, domain verification
NS	Authoritative name servers
PTR	Reverse lookup: IP → name
SOA	Zone information (serial, timers)
SRV	Service location (host + port)

Recursive query: "give me the final answer" (client → resolver). **Iterative query:** "tell me who to ask next" (resolver → root/TLD). **TTL** controls caching.

```
R1(config)# ip domain-lookup                  ! (default) resolve names typed on the CLI
R1(config)# ip name-server 192.168.1.53 8.8.8.8
R1(config)# ip domain-name lab.local
R1(config)# ip host SW1 10.0.0.2              ! static host entry
R1# show hosts
```

14.3 NAT (Network Address Translation)

Translates private addresses to public ones at the network edge (saves public IPv4 addresses, hides internal addressing).

Term	Meaning	Example
Inside local	Real (private) IP of the inside host	192.168.10.5
Inside global	Public IP representing the inside host to the outside	203.0.113.10
Outside global	Real public IP of the outside host	142.250.1.1

Term	Meaning	Example
Outside local	How the outside host appears inside (usually = outside global)	142.250.1.1

Type	Mapping	Use
Static NAT	One private ↔ one public, permanent, works in both directions	Servers that must be reachable from the internet
Dynamic NAT	Private addresses take a free address from a public **pool** (one-to-one while in use)	Rare today
PAT / NAT overload	Many private addresses share **one** public IP (or a few), distinguished by **source port**	Almost every home and office internet connection

```
R1(config)# interface g0/0
R1(config-if)# ip nat inside
R1(config)# interface g0/1
R1(config-if)# ip nat outside

! PAT using the outside interface's address
R1(config)# access-list 1 permit 192.168.0.0 0.0.255.255  ! which inside hosts are translated
R1(config)# ip nat inside source list 1 interface g0/1 overload

! Dynamic NAT with a pool (add "overload" to make it PAT with a pool)
R1(config)# ip nat pool PUBLIC 203.0.113.20 203.0.113.30 netmask 255.255.255.224
R1(config)# ip nat inside source list 1 pool PUBLIC

! Static NAT for a web server
R1(config)# ip nat inside source static 192.168.10.5 203.0.113.10

R1# show ip nat translations
Pro Inside global        Inside local        Outside local      Outside global
tcp 203.0.113.2:51001    192.168.1.10:51001  142.250.1.1:443    142.250.1.1:443
--- 203.0.113.10         192.168.10.5        ---                ---
R1# show ip nat statistics
R1# clear ip nat translation *
```

NAT troubleshooting: inside/outside reversed, ACL doesn't match the inside subnet, no route to the internet (default route), the return path from the ISP doesn't point to the inside global addresses, pool exhausted (dynamic NAT without overload).

14.4 NTP (Network Time Protocol)

Accurate time is essential: log correlation across devices, certificate validity, authentication protocols, scheduled tasks, forensics.

Concept	Meaning
Stratum	Distance from a reference clock: stratum 0 = atomic/GPS clock, stratum 1 = server attached to it, each hop +1. Lower is better; 16 = unsynchronized
Client / server	Devices sync from servers; a router can be both (client upstream, server for the LAN)
Peer (symmetric)	Two devices sync with each other
Authentication	Keys prevent spoofed time sources

```
R1(config)# ntp server 192.0.2.10 prefer
R1(config)# ntp server 192.0.2.11
R1(config)# clock timezone IST 5 30
R1(config)# ntp authenticate
R1(config)# ntp authentication-key 1 md5 <key>
R1(config)# ntp trusted-key 1
R1(config)# ntp server 192.0.2.10 key 1
SW1(config)# ntp server 10.0.0.1                ! switches sync from the core router
R1(config)# ntp master 4  ! lab only: act as an authoritative stratum 4 source
R1# show ntp status                             ! "Clock is synchronized, stratum 3..."
R1# show ntp associations
R1# show clock detail
```

14.5 SNMP (Simple Network Management Protocol)

Component	Role
NMS / manager	Monitoring system (Zabbix, PRTG, LibreNMS, SolarWinds) that polls devices and receives notifications
Agent	Software on the managed device
MIB	Structured database of manageable objects, each identified by an **OID** (e.g. interface counters, CPU)

Operation	Direction	Purpose
Get / GetNext / GetBulk	Manager → agent (UDP 161)	Read values
Set	Manager → agent	Change a value (rarely enabled)
Trap	Agent → manager (UDP 162)	Unacknowledged event notification
Inform	Agent → manager	Acknowledged notification (resent if not confirmed)

Version	Security
SNMPv1 / v2c	**Community strings** sent in plain text ("public"/"private" defaults): read-only at most, ACL-restricted
SNMPv3	Users with **authentication** (SHA) and **encryption** (AES): security level `authPriv`

```
R1(config)# snmp-server group MONITOR v3 priv
R1(config)# snmp-server user nms MONITOR v3 auth sha <auth-pass> priv aes 128 <priv-pass>
R1(config)# snmp-server host 10.0.0.60 version 3 priv nms
R1(config)# snmp-server enable traps
```

14.6 Syslog

Devices generate log messages and can send them to the console, the terminal (SSH session with `terminal monitor`), a local **buffer**, and a central **syslog server** (UDP 514).

```
*Oct 18 10:15:22.123: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to down
 timestamp            facility-severity-mnemonic: description
```

Level	Name	Level	Name
0	**Emergency**	4	**Warning**

Level	Name	Level	Name
1	**Alert**	5	**Notification**
2	**Critical**	6	**Informational**
3	**Error**	7	**Debugging**

Memory trick: **E**very **A**wesome **C**isco **E**ngineer **W**ill **N**eed **I**ce-cream **D**aily. Configuring a level sends that level **and all more severe (lower-numbered)** levels.

```
R1(config)# service timestamps log datetime msec localtime show-timezone
R1(config)# logging buffered 64000 informational        ! local buffer (show logging)
R1(config)# logging host 10.0.0.60
R1(config)# logging trap warnings                       ! to the server: levels 0–4
R1(config)# logging console critical                    ! less noise on the console
R1(config)# logging source-interface loopback0
R1# show logging
```

14.7 QoS (Quality of Service)

When a link is congested, packets wait in queues or are dropped. QoS decides **which traffic gets priority**.

Traffic	Needs	Typical targets
Voice	Low, steady bandwidth; very sensitive to delay, jitter, loss	One-way delay < 150 ms, jitter < 30 ms, loss < 1%
Video (interactive)	More bandwidth, sensitive to delay/loss	Delay < 200–400 ms, loss < 0.1–1%
Data	Bursty; TCP retransmits losses	Business-critical vs best effort vs scavenger

QoS per-hop behaviors (the toolset)

Tool	What it does
Classification	Identify traffic: ACLs, interface, NBAR (application recognition), existing markings
Marking	Write the class into the packet: **DSCP** (6 bits in the IP header), **CoS / PCP** (3 bits in the 802.1Q tag)
Trust boundary	Where you start trusting markings (usually the IP phone or access switch); re-mark untrusted traffic from PCs
Queuing	Separate queues per class; **CBWFQ** guarantees bandwidth per class; **LLQ** adds a strict-priority queue for voice (policed so it can't starve others)
Congestion avoidance	**WRED** drops some lower-priority TCP packets early so queues don't fill (avoids tail drop and TCP global synchronization)
Policing	Enforces a rate by **dropping or re-marking** excess traffic (no delay added; used inbound and by ISPs)
Shaping	**Buffers** excess traffic to send it later at the configured rate (adds delay, smooths bursts; used outbound to match a provider's contracted rate)

DSCP marking	Value	Typical use
EF (Expedited Forwarding)	46	Voice
AF41	34	Interactive video
AF31	26	Call signaling / critical data
AF21	18	Transactional data
CS6	48	Network control (routing protocols)
Default (BE)	0	Best effort
CS1	8	Scavenger (backups, bulk)

AF classes (AFxy): x = class 1–4 (higher = better queue), y = **drop precedence** 1–3 (higher = dropped first).

14.8 File transfer for devices

Protocol	Port	Security	Use
TFTP	UDP 69	None (no authentication)	Simple, lab and IP phone provisioning, image transfers in trusted networks
FTP	TCP 20/21	Username/password in plain text	Legacy transfers
SCP / SFTP	TCP 22	Encrypted (SSH)	**Production** config backups and image upgrades

```
R1# copy running-config tftp://10.0.0.50/r1-backup.cfg
R1# copy scp://admin@10.0.0.50/images/new.bin flash:
R1(config)# ip ftp username backup
R1(config)# ip scp server enable                ! allow SCP copies to the router
```

14.9 CDP and LLDP

Discover directly connected neighbors: device ID, platform, connected port, IP address, IOS version, VLAN information.

	CDP	LLDP
Standard	Cisco	IEEE 802.1AB (multi-vendor)
Default	Enabled on Cisco devices	Often disabled on Cisco: `lldp run`
Timers	60 s advertisements, 180 s hold	30 s, 120 s hold
Extensions	Voice VLAN for Cisco phones	LLDP-MED for phones (voice VLAN, PoE)

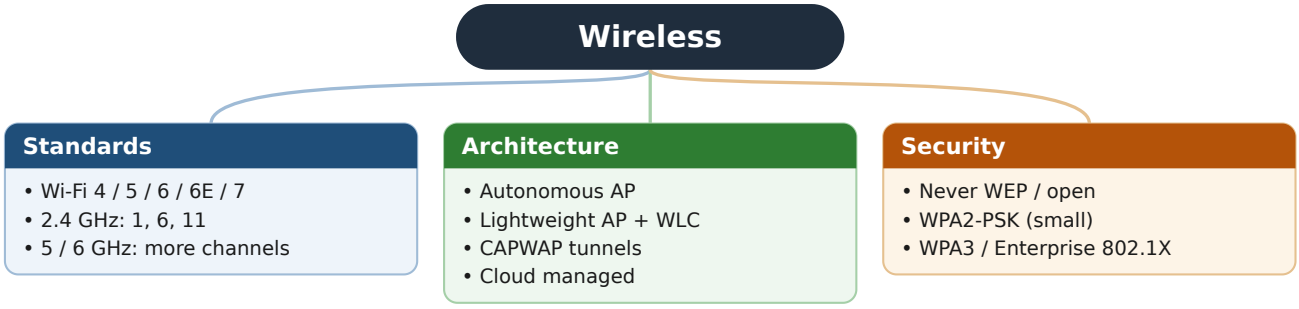
```
SW1# show cdp neighbors
Device ID   Local Intrfce   Holdtme   Capability   Platform   Port ID
R1          Gig 0/1         150       R S I        ISR4331    Gig 0/0/1
SW1# show cdp neighbors detail                   ! includes neighbor IP and IOS version
SW1(config)# lldp run
SW1# show lldp neighbors detail
SW1(config)# interface g0/24                     ! internet-facing / untrusted port
SW1(config-if)# no cdp enable
SW1(config-if)# no lldp transmit
SW1(config-if)# no lldp receive
```

CDP/LLDP reveal network details: disable them on ports facing untrusted networks.

REMEMBER

- Syslog 0–7: **E**very **A**wesome **C**isco **E**ngineer **W**ill **N**eed **I**ce-cream **D**aily.
- Host shows **169.254.x.x** → DHCP failed (check relay, VLAN, server).

15. Wireless (Wi-Fi)



15.1 RF fundamentals

Wi-Fi uses **radio frequency (RF)** waves. Key properties:

Term	Meaning
Frequency	Oscillations per second (GHz). Higher frequency = more bandwidth available, **shorter range**, worse at penetrating walls
Wavelength	Distance of one cycle (2.4 GHz ≈ 12.5 cm)
Amplitude / power	Signal strength, in **dBm** (0 dBm = 1 mW; +3 dB ≈ double, −3 dB ≈ half, +10 dB = ×10)
RSSI	Received signal strength, e.g. −50 dBm excellent, −67 dBm good for voice/video, −80 dBm poor
Noise floor	Background RF energy, e.g. −95 dBm
SNR	Signal minus noise in dB (−65 − (−95) = 30 dB): higher is better; 25 dB+ for good performance
Behaviors	**Absorption** (walls, water, people), **reflection** (metal), **refraction**, **diffraction**, **scattering**
Interference	Co-channel (same channel, nearby APs take turns), adjacent-channel (overlapping channels: worse), non-Wi-Fi (microwaves, Bluetooth, cordless phones)

Wi-Fi is a **shared, half-duplex medium**: only one device transmits on a channel at a time. It uses **CSMA/CA** (collision *avoidance*: listen, wait a random back-off, transmit, expect an ACK) because radios can't detect collisions while transmitting.

15.2 Bands, channels and standards

Band	Range	Interference	Non-overlapping channels
2.4 GHz	Longest	High (Bluetooth, microwaves, neighbors)	**1, 6, 11** (20 MHz channels; in some countries 1, 5, 9, 13)
5 GHz	Shorter	Lower	About 20–25 × 20 MHz channels (some need **DFS**: radar avoidance)
6 GHz	Shortest	Lowest (new, clean)	Up to 59 × 20 MHz or 7 × 160 MHz; Wi-Fi 6E/7 devices only; WPA3 required

Channel width: 20, 40, 80, 160 (and 320 in Wi-Fi 7) MHz. Wider = faster per client, but fewer non-overlapping channels: enterprises often use 20/40 MHz in dense areas.

IEEE standard	Wi-Fi name	Bands	Max data rate (theoretical)	Key features
802.11b/a/g	—	2.4 / 5 / 2.4	11 / 54 / 54 Mbps	Legacy
802.11n	Wi-Fi 4	2.4 / 5	600 Mbps	MIMO, 40 MHz
802.11ac	Wi-Fi 5	5	~3.5 Gbps (wave 2: ~6.9)	80/160 MHz, MU-MIMO downlink
802.11ax	**Wi-Fi 6 / 6E**	2.4 / 5 / (6E: 6)	~9.6 Gbps	**OFDMA**, uplink MU-MIMO, BSS coloring, target wake time: efficiency in dense areas
802.11be	Wi-Fi 7	2.4 / 5 / 6	~46 Gbps	320 MHz, multi-link operation (MLO), 4K-QAM

15.3 Wireless terminology

Term	Meaning
SSID	Network name clients see (up to 32 characters). One AP can broadcast several SSIDs, each mapped to a VLAN
BSSID	MAC address of an AP radio for one SSID: identifies the cell
BSS	One AP and its associated clients (the cell, Basic Service Set)
ESS	Several APs with the same SSID connected by a wired network: clients **roam** between them
IBSS	Ad hoc: devices directly, no AP
Distribution system (DS)	The wired network connecting APs
Repeater / workgroup bridge / outdoor bridge	Extend coverage / connect a wired device via Wi-Fi / connect buildings
Mesh	APs connect to each other wirelessly; only some (root APs) are wired
Association	Client joins an AP: probe → authentication → association → (802.1X/PSK key exchange)
Roaming	Client moves between APs in an ESS; fast roaming (802.11r) helps voice

15.4 Wireless architectures

Architecture	How	Where
Autonomous APs	Each AP configured and managed individually; bridges SSIDs to VLANs itself	Very small sites
Lightweight APs + WLC (split-MAC)	APs handle real-time functions (beacons, ACKs, encryption); the **Wireless LAN Controller** handles management, security policy, RF management, roaming. APs and WLC talk over **CAPWAP** tunnels	Campus networks

Architecture	How	Where
Cloud-managed	APs managed from a cloud dashboard (Cisco Meraki, Mist); data stays local	Distributed organizations, simple operations
Embedded / Mobility Express	WLC function runs on a switch or on one AP	Small/medium sites

CAPWAP (Control And Provisioning of Wireless Access Points): **UDP 5246** control (encrypted with DTLS) and **UDP 5247** data (client traffic tunneled to the WLC, optionally encrypted). The AP discovers the WLC (DHCP option 43, DNS `CISCO-CAPWAP-CONTROLLER`, broadcast, previously learned), then joins and downloads its configuration.

WLC deployment models: **centralized** (one WLC cluster in the data center), **distributed**, **cloud-based**, **embedded** in switches.

15.5 Lightweight AP modes

Mode	Purpose
Local (default)	Serves clients; all client traffic tunneled to the WLC via CAPWAP; scans other channels briefly for monitoring
FlexConnect	For branches: AP switches client traffic **locally** to VLANs (no tunnel to a distant WLC) and keeps working if the WAN link to the WLC fails
Monitor	No clients; dedicated scanning for rogue APs, wIPS, location
Sniffer	Captures 802.11 frames and sends them to a Wireshark station
Rogue detector	Listens on the wired network for MACs of rogue APs
SE-Connect	Spectrum analysis (finds non-Wi-Fi interference)
Bridge / Mesh	Point-to-point/multipoint bridging and mesh backhaul
Flex+Bridge	FlexConnect with mesh

15.6 Wired connections for WLAN components

```
                  WLC ── trunk (802.1Q, often LAG/EtherChannel) ── Core/Distribution switch
                                                                          |
                                                                  Access switch ── trunk
                                                                      |
Lightweight AP (Local mode) ── ACCESS port (mgmt VLAN; CAPWAP carries client VLANs)
FlexConnect AP ── TRUNK port (switches client SSIDs to local VLANs)
Autonomous AP  ── TRUNK port (one VLAN per SSID)
```

WLC port / interface	Meaning
Service port	Out-of-band management (bootstrap, recovery)
Distribution system port	Normal traffic, connected as an 802.1Q **trunk** to the switch; ports can be bundled with **LAG** (static EtherChannel, mode `on`: the WLC doesn't negotiate LACP/PAgP)
Management interface	WLC management IP (GUI/SSH) and CAPWAP termination

WLC port / interface	Meaning
Dynamic interfaces	One per client VLAN (maps WLANs/SSIDs to VLANs)
Virtual interface	Used for web authentication redirect and DHCP relay (non-routable IP)
Redundancy port	For HA pairs

APs are powered by **PoE** (802.3at/bt) from the access switch.

15.7 Wireless security

Method	Encryption / authentication	Status
Open	None	Only for guest captive portals (better: OWE "Enhanced Open")
WEP	RC4, static keys	**Broken: never use**
WPA	TKIP	Obsolete
WPA2-Personal (PSK)	AES-CCMP, shared passphrase	OK for home/small office; everyone shares one key
WPA2-Enterprise	AES-CCMP + **802.1X/EAP** with a **RADIUS** server: per-user credentials or certificates	Standard in companies
WPA3-Personal	**SAE** (Simultaneous Authentication of Equals): resists offline dictionary attacks, forward secrecy	Recommended for PSK networks
WPA3-Enterprise	802.1X, optional 192-bit mode	Recommended in enterprises

Also required with WPA3: **Protected Management Frames (PMF, 802.11w)**, which stops deauthentication attacks.

802.1X roles: supplicant (client), **authenticator** (AP/WLC or switch), **authentication server** (RADIUS: Cisco ISE, FreeRADIUS, Microsoft NPS). EAP methods: EAP-TLS (certificates on both sides: strongest), PEAP-MSCHAPv2 (username/password inside a TLS tunnel), EAP-FAST.

Other wireless security practices: separate guest SSID in its own VLAN with internet-only access, client isolation on guest WLANs, rogue AP detection, strong unique passphrases rotated when staff leave, disable WPS.

15.8 Configuring a WLAN on a Cisco WLC (GUI, WPA2-PSK)

Typical steps on an AireOS/Catalyst 9800-style WLC GUI:

1. **Create the dynamic interface / VLAN mapping:** Controller → Interfaces → New: name `sales`, VLAN 10, IP address in VLAN 10, gateway, DHCP server address (the WLC relays DHCP).
2. **Create the WLAN:** WLANs → Create New: type WLAN, **profile name**, **SSID** `Corp-Sales`, WLAN ID.
3. **General tab:** status **Enabled**, interface/interface group = `sales` (the VLAN), broadcast SSID on.
4. **Security tab → Layer 2:** security **WPA+WPA2** (or WPA2+WPA3), **WPA2 policy with AES** encryption, authentication key management **PSK**, set the **pre-shared key** (ASCII passphrase). For Enterprise: **802.1X** and add RADIUS servers under Security → AAA.
5. **QoS tab:** QoS profile (Platinum = voice, Gold = video, Silver = best effort, Bronze = background).

6. **Advanced tab:** session timeout, client load balancing/band select, **FlexConnect local switching** (for branch APs), DHCP required, client exclusion.
7. **Apply** and verify: Monitor → Clients (associated clients, IP, VLAN, RSSI/SNR).

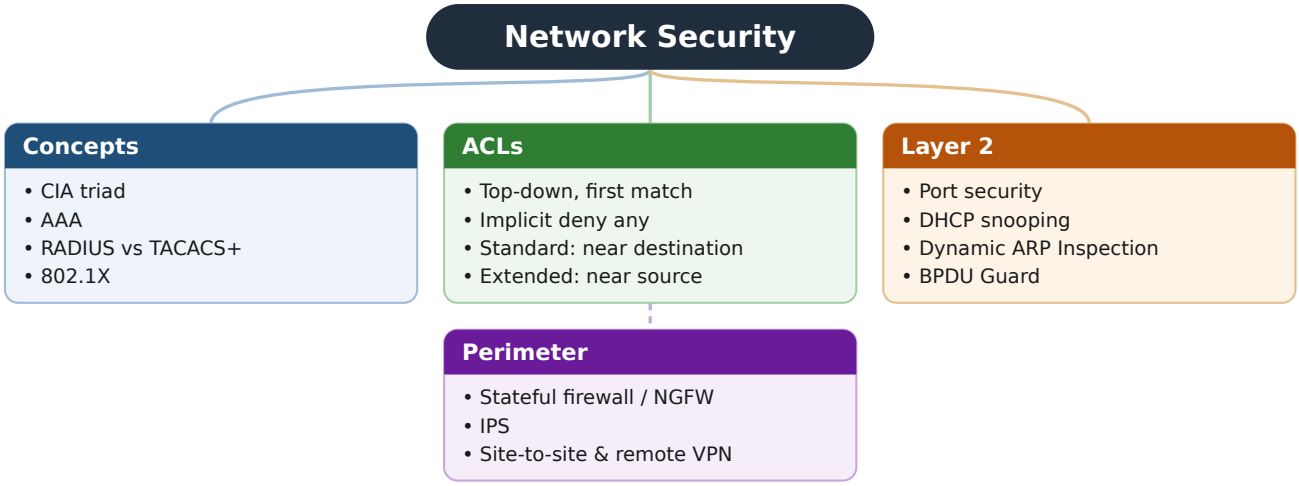
15.9 Wireless troubleshooting

Symptom	Check
Can't see the SSID	WLAN enabled and broadcast, AP joined to the WLC, radio enabled, client supports the band (6 GHz)
Associates but no IP	VLAN/interface mapping, trunk allows the VLAN, DHCP scope/relay
Authentication fails	PSK mismatch, RADIUS reachability/shared secret, certificate trust (EAP-TLS/PEAP), WPA version incompatibility
Slow or drops	Low RSSI/SNR (coverage holes), co-channel interference (too many APs on the same channel, wide channels), non-Wi-Fi interference, sticky clients not roaming, overloaded AP
AP won't join the WLC	IP/DHCP for the AP, option 43/DNS discovery, routing/firewall for UDP 5246–5247, country code/regulatory domain mismatch, certificate/time on the WLC

REMEMBER

- 2.4 GHz = longer range, more interference. 5/6 GHz = faster, shorter range.
- Companies use **Enterprise (802.1X + RADIUS)**, not a shared password.

16. Network Security Fundamentals



16.1 Key concepts

Term	Meaning	Example
Vulnerability	A weakness	Unpatched IOS, Telnet enabled, weak password
Threat	A potential danger that could exploit a weakness	Attacker, malware, insider
Exploit	The method/tool that uses a vulnerability	A script abusing a bug
Mitigation	What reduces the risk	Patching, ACLs, SSH, MFA
Risk	Likelihood × impact	
CIA triad	Confidentiality, Integrity, Availability	
Defense in depth	Several layers of protection	Firewall + ACL + 802.1X + endpoint protection
Least privilege	Only the access needed	Read-only SNMP, per-role CLI privileges
Zero trust	Never trust by location; verify every user, device and request	ZTNA instead of flat VPN access

Security program elements

- **User awareness:** everyone knows the threats (phishing, tailgating, USB drops) and how to report them; regular phishing simulations.
- **Training:** role-specific technical training for IT/network staff (secure configuration, incident response).
- **Physical access control:** locked rooms and racks, badge access, cameras, visitor logs, secured console ports. Physical access usually means full access (password recovery).
- Policies and procedures: acceptable use, password policy, change management, incident response plan, backups.

16.2 Common attacks

Attack	Description	Defense
Phishing / spear phishing / vishing / smishing	Tricking users via email/phone/SMS	Awareness training, MFA, email filtering
Social engineering, tailgating	Manipulating people, following into secure areas	Training, physical controls
Malware: virus, worm, trojan, ransomware	Malicious software (worms self-propagate; ransomware encrypts data)	Patching, endpoint protection (EDR), segmentation, offline backups
DoS / DDoS	Flooding a service (SYN flood, amplification via DNS/NTP reflection)	Rate limiting, DDoS scrubbing services, anti-spoofing (uRPF)
Spoofing	Faking IP/MAC/ARP/DHCP identity	ACL anti-spoofing, DHCP snooping, DAI, IP Source Guard
Man-in-the-middle (on-path)	Intercepting/modifying traffic (ARP poisoning, rogue AP)	Encryption (TLS, SSH, VPN), DAI, 802.1X, WPA3
Reconnaissance	Scanning to map the network (ping sweeps, port scans)	Firewalls, IPS, minimal exposed services
MAC flooding	Overflowing the switch MAC table so it floods all frames (sniffing)	Port security
DHCP starvation / spoofing	Exhausting the pool / rogue DHCP server handing out a malicious gateway	Port security, **DHCP snooping**
VLAN hopping	Switch spoofing via DTP, or double-tagging via the native VLAN	Disable DTP, static access ports, unused native VLAN
Password attacks	Brute force, dictionary, credential stuffing	Strong passwords, MFA, lockout/ `login block-for` , SSH keys
Buffer overflow / exploits	Abusing software bugs	Patching, IPS

16.3 Passwords and device access control

```
R1(config)# security passwords min-length 12
R1(config)# enable algorithm-type scrypt secret <strong-password>
R1(config)# username netadmin privilege 15 algorithm-type scrypt secret <strong-password>
R1(config)# username helpdesk privilege 1 algorithm-type scrypt secret <strong-password>
R1(config)# login block-for 300 attempts 5 within 60
R1(config)# login on-failure log
R1(config)# line console 0
R1(config-line)# login local
R1(config-line)# exec-timeout 5 0
R1(config)# line vty 0 15
R1(config-line)# login local
R1(config-line)# transport input ssh
R1(config-line)# exec-timeout 10 0
```

Password policy elements: minimum length (passphrases of 12–16+ characters beat complex short passwords), no reuse, no default/shared accounts, change on compromise (frequent forced rotation is no longer recommended by NIST), screen against breached-password lists, account lockout.

Alternatives and additions to passwords:

Method	Example
Multi-factor authentication (MFA)	Something you **know** (password) + **have** (phone app, hardware key) + **are** (biometric). Two different categories
Certificates	Digital certificates (PKI) for devices/users: 802.1X EAP-TLS, VPN, SSH certificates
Biometrics	Fingerprint, face recognition (usually as one MFA factor)
SSH keys	Public-key authentication for administrators/automation

16.4 AAA, RADIUS and TACACS+

AAA: Authentication (who are you?), **Authorization** (what may you do?), **Accounting** (what did you do: logged for audit). Centralized AAA means one place to add/remove admin access across hundreds of devices.

	RADIUS	TACACS+
Standard	Open (RFC 2865)	Cisco-developed (now RFC 8907)
Transport	UDP 1812/1813	**TCP 49**
Encryption	Only the password	**Entire payload**
AAA separation	Authentication and authorization combined	**Separate** (per-command authorization)
Typical use	**Network access**: 802.1X, Wi-Fi, VPN users	**Device administration**: who can log in to routers/switches and run which commands
Servers	Cisco ISE, FreeRADIUS, Microsoft NPS	Cisco ISE, tac_plus

```
R1(config)# aaa new-model
R1(config)# tacacs server ISE1
R1(config-server-tacacs)# address ipv4 10.0.0.70
R1(config-server-tacacs)# key <shared-secret>
R1(config)# aaa authentication login default group tacacs+ local  ! local = fallback
R1(config)# aaa authorization exec default group tacacs+ local
R1(config)# aaa authorization commands 15 default group tacacs+ local
R1(config)# aaa accounting commands 15 default start-stop group tacacs+
```

Always keep a **local fallback account** for when the AAA server is unreachable.

802.1X port-based access control: a switch port (or WLAN) passes only EAP authentication traffic until the device authenticates via RADIUS; then the port opens (optionally placing the device in a VLAN assigned by the server). Devices without a supplicant (printers, cameras) use **MAB** (MAC Authentication Bypass).

16.5 Access Control Lists (ACLs)

Rules

- An ACL is an ordered list of **permit/deny** statements (ACEs), checked **top to bottom: first match wins**, the rest is skipped.
- Every ACL ends with an invisible **implicit** `deny any`: an ACL with only deny statements blocks everything. Add `permit ip any any` (or `permit any`) when intended.

- Apply **one ACL per interface, per direction, per protocol** (IPv4/IPv6).
- Direction is from the **router's** point of view: **in** = arriving at the interface (filtered before routing), **out** = leaving it.
- Put specific statements before general ones.
- ACLs on an interface don't filter traffic **generated by the router itself**.

Type	Numbers	Matches	Placement
Standard	1–99, 1300–1999	**Source IP only**	As close to the **destination** as possible (otherwise it blocks the source from everything)
Extended	100–199, 2000–2699	Protocol, source, destination, ports, flags	As close to the **source** as possible (drop unwanted traffic early)
Named	A name	Standard or extended; editable by sequence number	Same rules

Wildcard masks

0 = must match, 1 = ignore. Wildcard = 255.255.255.255 − subnet mask.

Match	Address + wildcard	Keyword form
One host	`10.1.1.5 0.0.0.0`	`host 10.1.1.5`
A /24	`192.168.10.0 0.0.0.255`	
A /26	`192.168.10.64 0.0.0.63`	
Everything	`0.0.0.0 255.255.255.255`	`any`

Configuration examples

```
! Standard numbered: block 192.168.20.0/24 from the server LAN (placed near the destination)
R1(config)# access-list 10 deny 192.168.20.0 0.0.0.255
R1(config)# access-list 10 permit any
R1(config)# interface g0/2                              ! interface toward the server LAN
R1(config-if)# ip access-group 10 out
```

```
! Extended named: users may reach the web server only on HTTPS, plus DNS anywhere; log denies
R1(config)# ip access-list extended USERS-IN
R1(config-ext-nacl)# 10 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 443
R1(config-ext-nacl)# 20 deny ip 192.168.10.0 0.0.0.255 host 10.0.0.80 log
R1(config-ext-nacl)# 30 permit udp 192.168.10.0 0.0.0.255 any eq domain
R1(config-ext-nacl)# 40 permit tcp 192.168.10.0 0.0.0.255 any eq 443
R1(config-ext-nacl)# 50 permit icmp 192.168.10.0 0.0.0.255 any echo
R1(config-ext-nacl)# 60 deny ip any any log
R1(config)# interface g0/0                              ! interface where users' traffic enters
R1(config-if)# ip access-group USERS-IN in
```

```
! Edit a named ACL: insert and delete by sequence number
R1(config)# ip access-list extended USERS-IN
R1(config-ext-nacl)# 15 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 22
R1(config-ext-nacl)# no 50
R1# show access-lists USERS-IN                           ! hit counters per line
R1# show ip interface g0/0 | include access list
```

```
! Restrict management access (standard ACL on the VTY lines)
R1(config)# ip access-list standard MGMT
R1(config-std-nacl)# permit 10.10.10.0 0.0.0.255
R1(config)# line vty 0 15
R1(config-line)# access-class MGMT in
```

Port operators: `eq`, `neq`, `gt`, `lt`, `range 20 21`. Keywords for well-known ports: `www` (80), `domain` (53), `ftp`, `telnet`, `smtp`. `established` matches TCP packets with ACK/RST set (return traffic of connections initiated from inside: a simple, non-stateful filter).

IPv6 ACLs: `ipv6 access-list NAME` and `ipv6 traffic-filter NAME in|out`; they include implicit permits for neighbor discovery before the final deny.

16.6 Layer 2 security

Port security

```
SW1(config)# interface f0/5
SW1(config-if)# switchport mode access                  ! port security needs a static mode
SW1(config-if)# switchport access vlan 10
SW1(config-if)# switchport port-security
SW1(config-if)# switchport port-security maximum 2      ! default 1
SW1(config-if)# switchport port-security mac-address sticky  ! learned MACs saved to config
SW1(config-if)# switchport port-security violation restrict  ! shutdown | restrict | protect
SW1(config-if)# switchport port-security aging time 60
SW1# show port-security
SW1# show port-security interface f0/5
SW1# show port-security address
```

Violation mode	Drops offending traffic	Log/SNMP + violation counter	Shuts the port (err-disabled)
shutdown (default)	Yes	Yes	**Yes**
restrict	Yes	Yes	No
protect	Yes	No	No

DHCP snooping

Stops **rogue DHCP servers** and DHCP starvation. Ports are **untrusted** by default: DHCP server messages (OFFER, ACK) arriving on untrusted ports are dropped. Mark only the uplinks toward legitimate DHCP servers as **trusted**. The switch builds a **binding table** (MAC, IP, VLAN, port, lease) used by DAI and IP Source Guard.

```
SW1(config)# ip dhcp snooping
SW1(config)# ip dhcp snooping vlan 10,20
SW1(config)# no ip dhcp snooping information option  ! if the server rejects option 82
SW1(config)# interface g0/1                          ! uplink toward the real DHCP server/relay
SW1(config-if)# ip dhcp snooping trust
SW1(config)# interface range f0/1 - 24
SW1(config-if-range)# ip dhcp snooping limit rate 10     ! DHCP packets per second (anti-starvation)
SW1# show ip dhcp snooping
SW1# show ip dhcp snooping binding
```

Dynamic ARP Inspection (DAI)

Stops **ARP spoofing/poisoning**: on untrusted ports, ARP packets are checked against the DHCP snooping binding table (or ARP ACLs for static hosts); invalid ones are dropped.

```
SW1(config)# ip arp inspection vlan 10,20
SW1(config)# ip arp inspection validate src-mac dst-mac ip
SW1(config)# interface g0/1
SW1(config-if)# ip arp inspection trust                  ! uplinks / trusted infrastructure
SW1# show ip arp inspection
SW1# show ip arp inspection statistics
```

Other Layer 2 protections: **IP Source Guard** (only allow the IP/MAC from the binding table on a port), **BPDU Guard** (10.6), storm control, private VLANs, shutting down unused ports and placing them in an unused "parking" VLAN.

Recover an err-disabled port: fix the cause → `shutdown` → `no shutdown`, or `errdisable recovery cause psecure-violation` with `errdisable recovery interval 300`.

16.7 Firewalls and IPS

Item	Meaning
Packet filter (ACL)	Stateless: each packet judged alone; return traffic needs its own rules
Stateful firewall	Tracks connections in a state table; return traffic for allowed sessions is permitted automatically
NGFW	Adds application visibility and control (App-ID), user identity, **IPS**, URL filtering, malware sandboxing, TLS decryption
Zones	Inside (trusted), outside (internet), **DMZ** (public-facing servers: reachable from outside, but can't freely reach inside)
IDS vs **IPS**	IDS monitors a copy of traffic and alerts; IPS is inline and blocks. Use **signatures** (known attacks) and anomaly detection

16.8 VPNs

VPN type	How	Use
Site-to-site IPsec	Routers/firewalls at each site build an encrypted tunnel over the internet; users need no client	Branch ↔ head office, office ↔ cloud (AWS Site-to-Site VPN)
Remote-access VPN	Each user's device runs a client and connects to a VPN gateway	Remote workers
TLS/SSL VPN	Remote access over TLS (443): clientless (browser portal) or client-based (Cisco Secure Client/AnyConnect)	Works through most firewalls
GRE	Generic tunnel; carries multicast/routing protocols but **no encryption**	Combined as **GRE over IPsec**
DMVPN	Cisco: dynamic spoke-to-spoke tunnels with a hub	Large branch networks
ZTNA	Per-application access after identity and device checks (no network-wide access)	Modern replacement for full-tunnel VPN

IPsec building blocks:

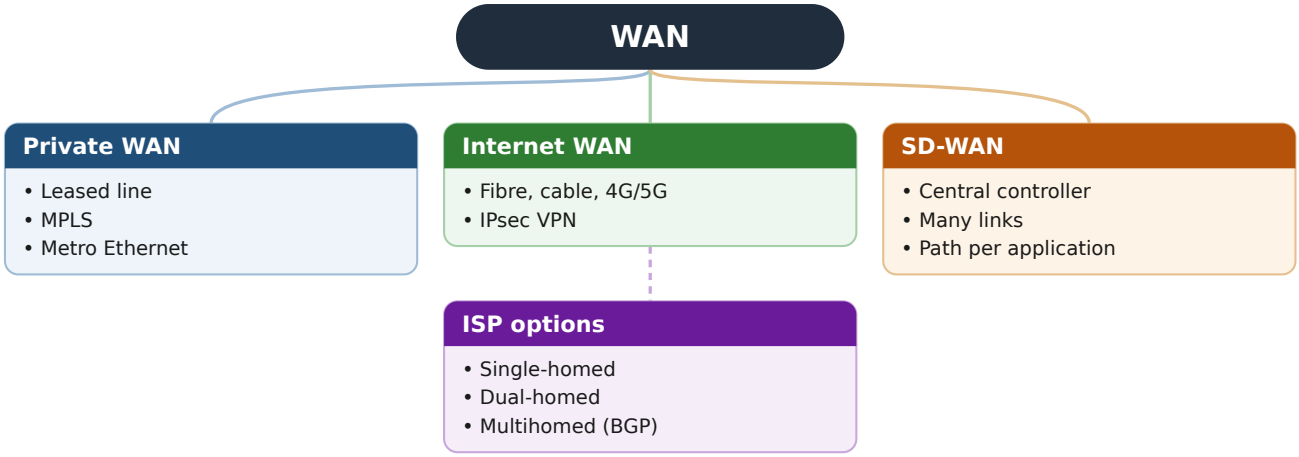
- **IKE** (IKEv2 preferred) negotiates security parameters and keys: authenticates peers (pre-shared key or certificates), establishes the secure channel (phase 1 / IKE SA), then the data tunnels (phase 2 / IPsec SAs).
- **ESP** (IP protocol 50): encryption (AES) + integrity (SHA) + authentication; **AH** (protocol 51) integrity only (rare; breaks with NAT).
- **Tunnel mode** encrypts the whole original packet and adds a new IP header (site-to-site); **transport mode** protects only the payload (host-to-host).
- NAT traversal: ESP inside UDP 4500 when NAT is in the path.

16.9 Device hardening checklist

- `enable secret` (scrypt), local users with `secret`, AAA with TACACS+ and local fallback.
- SSH v2 only, `transport input ssh`, VTY `access-class`, `exec-timeout`, `login block-for`.
- Disable unused services: HTTP server (or HTTPS only), CDP/LLDP on edge ports, `no ip source-route`, `no service pad`, `no ip proxy-arp` where not needed.
- Shut down unused ports and put them in an unused VLAN; port security, DHCP snooping, DAI, BPDU Guard on access ports.
- NTP with authentication; syslog to a central server; SNMPv3 only.
- Login banner (legal notice); regular configuration backups; keep IOS patched (check vendor security advisories).
- Management plane on a separate management network/VRF; control plane policing (CoPP) on core devices.

> **REMEMBER**
> - Every ACL ends with an invisible **deny any**.
> - Place **standard ACLs near the destination**, **extended ACLs near the source**.
> - Wildcard mask = 255.255.255.255 − subnet mask.

17. WAN & Internet Connectivity



17.1 WAN technologies

Technology	Description	Pros / cons
Leased line	Dedicated private circuit between two sites (e.g. E1/T1, Ethernet private line)	Reliable, fixed bandwidth / expensive, point-to-point only
MPLS Layer 3 VPN	Provider network connects all company sites; the provider's PE routers exchange your routes (CE ↔ PE routing via static, OSPF or BGP); **labels** forward traffic inside the provider	Any-to-any connectivity, QoS, SLAs / costly, slow to provision
Metro Ethernet (Carrier Ethernet)	Ethernet service from a provider: E-Line (point-to-point), E-LAN (multipoint, like one big switch), E-Tree (hub and spoke)	Simple Ethernet handoff, high bandwidth in cities
Broadband internet	Fibre (FTTH/GPON), cable (DOCSIS), DSL, fixed wireless	Cheap, high bandwidth / best effort, no SLA
4G / 5G cellular	Mobile data with a SIM in the router	Quick to deploy, good backup link / variable performance, data caps
Satellite	GEO (high latency ~600 ms) or LEO constellations (Starlink: ~25–50 ms)	Remote sites
Internet VPN	IPsec tunnels over normal internet links (16.8)	Cheap, encrypted / internet performance
SD-WAN	Controller-managed overlay across several transports (MPLS, internet, 5G): central policy, application-aware path selection, encryption, zero-touch branch provisioning	Lower cost, agility / new skills and platform

WAN topologies

Point-to-point, **hub-and-spoke** (branches through a central hub: simple, hub is a bottleneck/SPOF), full mesh, partial mesh, dual-homed.

Connecting to the internet

Option	Meaning
Single-homed	One link to one ISP
Dual-homed	Two links to the **same** ISP
Multihomed	Links to **two different** ISPs
Dual-multihomed	Two links to each of two ISPs

Internet edge design: edge router/firewall, **PAT** for users, static NAT for published servers, default route to the ISP (floating static for a backup link), **BGP** when multihomed with your own public prefix so the internet can reach you via either ISP.

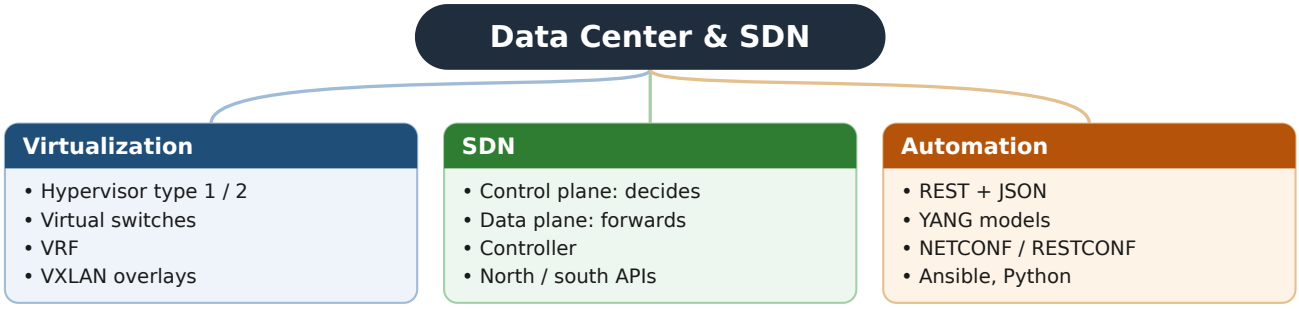
PPPoE (for DSL)

```
R1(config)# interface g0/1
R1(config-if)# pppoe enable
R1(config-if)# pppoe-client dial-pool-number 1
R1(config)# interface dialer 1
R1(config-if)# encapsulation ppp
R1(config-if)# ip address negotiated
R1(config-if)# ip mtu 1492                          ! 8 bytes of PPPoE overhead
R1(config-if)# dialer pool 1
R1(config-if)# ppp chap hostname <username>
R1(config-if)# ppp chap password <password>
R1(config)# ip route 0.0.0.0 0.0.0.0 dialer 1
```

REMEMBER

- SD-WAN = cheaper links + central control + automatic best-path selection.

18. Data Center, Virtualization, SDN & Automation



18.1 Virtualization

Term	Meaning
Virtual machine (VM)	A complete computer (own OS kernel) running on virtualized hardware
Hypervisor	Runs VMs. **Type 1** runs on bare metal (VMware ESXi, KVM, Microsoft Hyper-V, Xen): data centers. **Type 2** runs on a host OS (VirtualBox, VMware Workstation): laptops/labs
Virtual switch	Software switch connecting VMs to each other and to physical NICs (VMware vSwitch/vDS, Linux bridge, Open vSwitch); supports VLAN tags, so the physical switch port toward a hypervisor is usually a **trunk**
Containers	Share the host kernel; lighter than VMs (milliseconds to start). Networking via bridges, NAT, overlays (Docker, Kubernetes CNI)
VRF	Virtual Routing and Forwarding: several independent routing tables on one router; overlapping IP ranges allowed (multi-tenant, management separation, MPLS VPNs)
VXLAN	Overlay that carries Layer 2 segments over a Layer 3 network in UDP 4789; 24-bit VNI = 16 million segments; used in data center fabrics and clouds
Cloud	Virtualized resources on demand (IaaS, PaaS, SaaS); public, private, hybrid (chapter 19)

```
! VRF example: separate customer routing tables on one router (VRF-lite)
R1(config)# vrf definition CUST-A
R1(config-vrf)# address-family ipv4
R1(config)# interface g0/1
R1(config-if)# vrf forwarding CUST-A           ! removes the IP: re-enter it afterwards
R1(config-if)# ip address 10.1.1.1 255.255.255.0
R1# show ip route vrf CUST-A
R1# ping vrf CUST-A 10.1.1.10
```

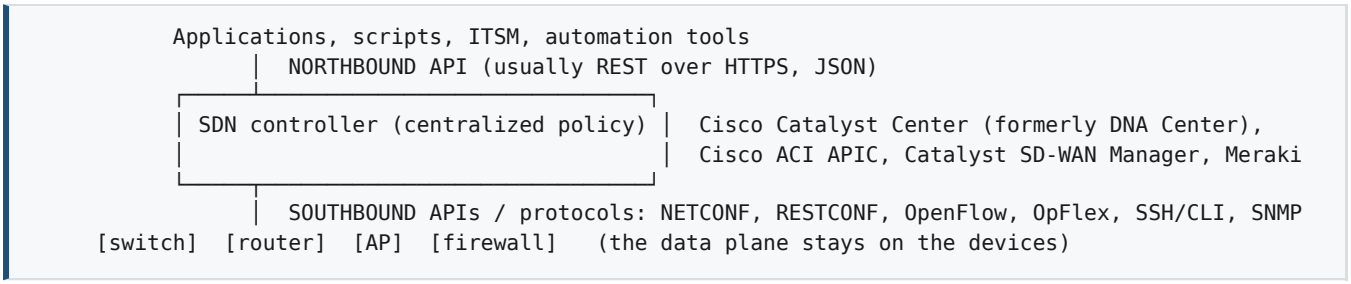
18.2 Planes of a network device

Plane	Job	Examples
Data (forwarding) plane	Moves packets/frames: lookup, rewrite, forward, filter	Forwarding with CEF tables, ACL checks, NAT, encapsulation

Plane	Job	Examples
Control plane	Builds the information the data plane uses	OSPF, BGP, STP, ARP, MAC learning
Management plane	How people/systems manage the device	SSH, SNMP, syslog, NTP, APIs

Traditional networking: every device has its own control plane and is configured one by one through the CLI.

18.3 Software-Defined Networking (SDN) and controllers



Term	Meaning
Controller-based networking	A central controller holds the intent/policy and programs many devices (some control-plane functions centralized)
Northbound interface (NBI)	Between the controller and applications above it (REST APIs)
Southbound interface (SBI)	Between the controller and network devices
Underlay	The physical IP network that provides connectivity (routing, links)
Overlay	Virtual networks built on top with tunnels (VXLAN, GRE, IPsec, LISP)
Fabric	The underlay + overlay + control plane managed as one system
Intent-based networking (IBN)	You declare business intent ("contractors can reach only the internet"); the controller translates it to configs and continuously verifies it
Cisco SD-Access	Campus fabric: IS-IS underlay, LISP control plane, VXLAN data plane, Cisco TrustSec (SGTs) policy, managed by Catalyst Center

Traditional vs controller-based

	Traditional	Controller-based
Configuration	Device by device (CLI)	Centrally, by policy
Consistency	Prone to drift and typos	Templates enforced everywhere
Speed of change	Slow for many devices	Fast, automated
Visibility	Per device	Network-wide, with assurance/analytics
Skills	CLI expertise	APIs, data models, automation, plus networking fundamentals

18.4 Why automation

Fewer manual errors (most outages are caused by changes), consistent configurations, faster changes and provisioning, compliance checks at scale, an audit trail in Git, and engineers freed from repetitive work. Automation also enables **network as code**: configurations in Git, reviewed by pull request, tested in CI, deployed by tools.

18.5 REST APIs

REST (Representational State Transfer) APIs expose **resources** (URLs) manipulated with HTTP methods. Stateless: each request carries everything needed (including authentication).

CRUD	HTTP method	Example	Typical success code
Create	`POST`	Create a VLAN	201 Created
Read	`GET`	List interfaces	200 OK
Update	`PUT` (replace) / `PATCH` (modify part)	Change a description	200 OK / 204 No Content
Delete	`DELETE`	Remove a VLAN	200 OK / 204 No Content

Status code	Meaning
200 OK / 201 Created / 204 No Content	Success
400 Bad Request	Malformed request or invalid data
401 Unauthorized	Missing/invalid authentication
403 Forbidden	Authenticated, but not allowed
404 Not Found	Resource doesn't exist
500 Internal Server Error	Server-side failure

Authentication types: HTTP **Basic** (username:password base64-encoded in the `Authorization` header: only over HTTPS), **API key** (static key in a header), **bearer token** (obtain a token by logging in once, then send `Authorization: Bearer <token>`; tokens expire), **OAuth 2.0** (delegated, scoped tokens).

Data encoding: JSON (most REST APIs), XML (NETCONF, older APIs), YAML (humans/Ansible). Headers declare it: `Content-Type: application/json`, `Accept: application/json`.

```
GET https://controller.example.com/api/v1/network-device?hostname=SW1
Authorization: Bearer eyJhbGciOi...
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
{"response": [{"hostname": "SW1", "managementIpAddress": "10.0.0.2", "softwareVersion": "17.9.4"}]}
```

18.6 JSON

```
{
  "device": {
    "hostname": "SW1",
    "managed": true,
    "uptime_days": 41,
    "location": null,
    "interfaces": [
      {"name": "GigabitEthernet0/1", "enabled": true, "vlan": 10, "description": "Sales PC"},
      {"name": "GigabitEthernet0/2", "enabled": false, "vlan": 20, "description": "Spare"}
    ],
    "allowed_vlans": [10, 20, 100]
  }
}
```

JSON element	Example	Rules
Object	`{"key": value, ...}`	Unordered key/value pairs in curly braces; keys are strings in **double quotes**; pairs separated by commas
Array	`[10, 20, 100]`	Ordered list in square brackets
String	`"SW1"`	Double quotes only
Number	`41`, `3.5`	No quotes
Boolean	`true`, `false`	Lowercase, no quotes
Null	`null`	Lowercase

Common mistakes: single quotes, trailing comma after the last item, missing commas, comments (JSON has none). Whitespace doesn't matter.

XML equivalent style: `<hostname>SW1</hostname>`; **YAML**: indentation-based (`hostname: SW1`).

18.7 Data models and network APIs

Item	Meaning
YANG	A data **modeling language** describing configuration and operational data (standard IETF/OpenConfig models and vendor-native models)
NETCONF	Protocol to get/edit configuration using YANG models, XML encoding, over **SSH port 830**; supports candidate configs, validation, commit/rollback, locking
RESTCONF	REST-style access to YANG data over **HTTPS**, JSON or XML encoding
gNMI / streaming telemetry	Devices push operational data continuously (instead of SNMP polling)

```
"""Read interfaces from an IOS XE device with RESTCONF (YANG ietf-interfaces model)."""
import os

import requests

HOST = "10.0.0.1"
URL = f"https://{HOST}/restconf/data/ietf-interfaces:interfaces"
HEADERS = {"Accept": "application/yang-data+json"}
AUTH = (os.environ["NET_USER"], os.environ["NET_PASS"])   # never hardcode credentials


def get_interfaces():
    # verify=True with a trusted CA bundle in production; lab devices often use self-signed certs
    response = requests.get(URL, headers=HEADERS, auth=AUTH, timeout=10,
                            verify=os.environ.get("NET_CA_BUNDLE", True))
    response.raise_for_status()
    for intf in response.json()["ietf-interfaces:interfaces"]["interface"]:
        print(f'{intf["name"]:<25} enabled={intf.get("enabled")}')


if __name__ == "__main__":
    get_interfaces()
```

18.8 Configuration management and automation tools

Tool	Model	Language	Agent	Notes
Ansible	**Push** from a control node	YAML playbooks	**Agentless** (SSH, NETCONF, APIs)	The most common for networks (`cisco.ios`, `arista.eos`, `junipernetworks.junos` collections)
Terraform	Declarative desired state with a state file	HCL	Agentless (provider APIs)	Cloud networks, SD-WAN/ACI/firewall providers; plan/apply workflow
Puppet	**Pull** (agents check in)	Puppet DSL (manifests)	Agent (or proxy for network devices)	Server configuration, some network support
Chef	Pull	Ruby (cookbooks, recipes)	Agent	Server configuration
Python libraries	Scripts	Python	—	**Netmiko** (SSH/CLI), **NAPALM** (multi-vendor abstraction), `ncclient` (NETCONF), `requests` (REST), **Nornir** (framework), pyATS (testing)

```
# Ansible: create VLANs and configure an access port on Cisco IOS switches
- name: Configure access switches
  hosts: access_switches
  gather_facts: false
  tasks:
    - name: Ensure VLANs exist
      cisco.ios.ios_vlans:
        config:
          - vlan_id: 10
            name: SALES
          - vlan_id: 20
            name: HR
        state: merged

    - name: Configure the access port for a Sales PC
      cisco.ios.ios_l2_interfaces:
        config:
          - name: GigabitEthernet1/0/5
            mode: access
            access:
              vlan: 10
        state: merged

    - name: Save the configuration when changed
      cisco.ios.ios_config:
        save_when: changed
```

```
"""Netmiko: run show commands on several devices; credentials from the environment."""
import os

from netmiko import ConnectHandler

DEVICES = ["10.0.0.1", "10.0.0.2"]

for host in DEVICES:
    params = {
        "device_type": "cisco_ios",
        "host": host,
        "username": os.environ["NET_USER"],
        "password": os.environ["NET_PASS"],
    }
    with ConnectHandler(**params) as conn:
        print(f"===== {host} =====")
        print(conn.send_command("show ip interface brief"))
```

18.9 AI and machine learning in network operations

Type	What it does	Network examples
Machine learning (ML)	Learns patterns from data instead of fixed rules	Baselines of "normal" traffic per site/time, anomaly detection
Predictive AI	Forecasts future events from historical and real-time data	Predicting link saturation, failing optics/hardware, Wi-Fi capacity issues before users notice; proactive rerouting
Generative AI	Creates new content (text, configs, code) from prompts	Natural-language assistants that explain issues, summarize logs, draft configurations or automation scripts, answer "why is this client slow?"
AIOps	AI applied to IT operations data	Event correlation (one root cause instead of 500 alarms), noise reduction, automated remediation

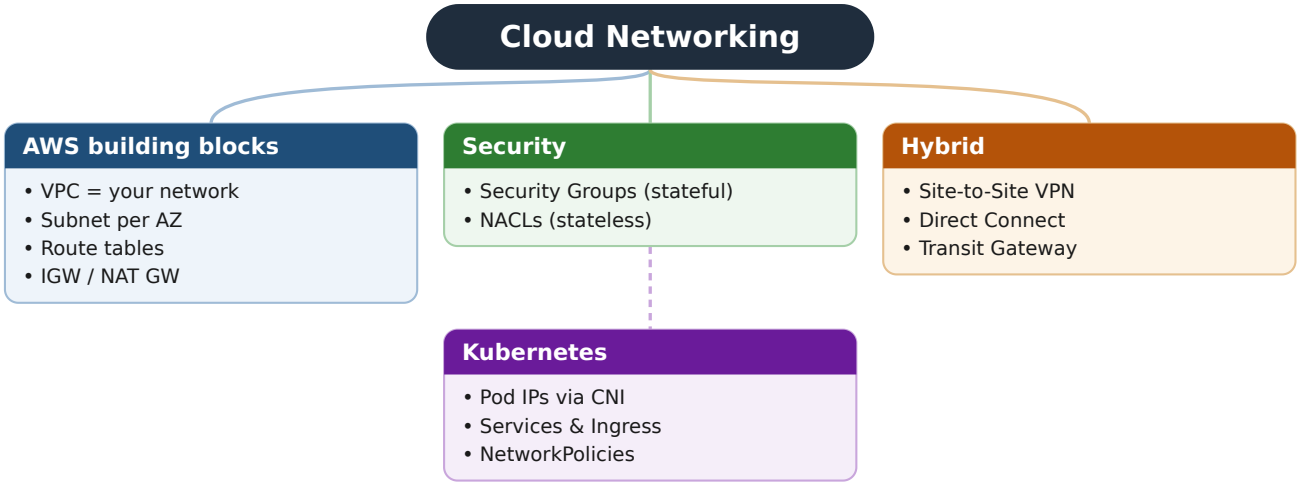
Examples: Cisco AI Assistant/AI-driven assurance in Catalyst Center and Meraki, Juniper Mist AI (Marvis), AI features in SD-WAN analytics.

Engineering cautions: AI output must be **verified** (generated configs can be wrong or insecure), never paste secrets or sensitive configurations into public AI tools, keep a human approval step for changes, and make sure the training data (telemetry) is good, because poor data gives poor predictions.

REMEMBER

- **Control plane** decides the path; **data plane** moves the packets; **management plane** is how you configure it.
- CRUD ↔ HTTP: Create = POST · Read = GET · Update = PUT/PATCH · Delete = DELETE.

19. Cloud Networking (AWS mapping)



19.1 Traditional network → AWS

Traditional network	AWS equivalent	Notes
Company network / data center	**VPC**	Your private IP space in one region (e.g. 10.0.0.0/16)
Subnet / VLAN	**Subnet**	Lives in **one Availability Zone**; AWS reserves 5 addresses per subnet (network, VPC router, DNS, future, broadcast)
Router + routing table	VPC router + **route tables**	Every subnet is associated with one route table; the `local` route covers the VPC
Internet edge router	**Internet Gateway (IGW)**	A subnet is "public" when its route table has `0.0.0.0/0 → igw-...`
NAT/PAT router	**NAT Gateway**	Outbound-only internet for private subnets; one per AZ for resilience
Stateful firewall on each host	**Security Group**	Instance/ENI level, allow rules only, stateful, can reference other SGs
Stateless ACL on a subnet	**Network ACL**	Subnet level, numbered allow/deny rules, stateless (allow ephemeral return ports)
Load balancer	**ALB** (Layer 7) / **NLB** (Layer 4) / GWLB (inline appliances)	
DNS server	**Route 53** (public and private hosted zones) + Route 53 Resolver (hybrid DNS endpoints)	
Site-to-site VPN	**AWS Site-to-Site VPN**	Two IPsec tunnels per connection, static or **BGP** routing
Leased line / MPLS to the cloud	**AWS Direct Connect**	Private dedicated connection, BGP, consistent latency

Traditional network	AWS equivalent	Notes
Core router connecting many networks	**Transit Gateway**	Hub for VPCs, VPNs and Direct Connect, with its own route tables
Point-to-point network link	**VPC peering**	One-to-one, **non-transitive**, no overlapping CIDRs
Private access to services	**VPC endpoints** (gateway for S3/DynamoDB; interface endpoints via PrivateLink)	Traffic stays on the AWS network, no NAT needed
SPAN port / NetFlow	**VPC Traffic Mirroring** / **VPC Flow Logs**	
IPAM spreadsheet	VPC IPAM	Plan and track CIDRs across accounts

19.2 Reference VPC design

```
VPC 10.0.0.0/16 (ap-south-1)        AZ a          AZ b          Default route
  Public (ALB, NAT GW)              10.0.0.0/24   10.0.1.0/24   0.0.0.0/0 → IGW
  Private app (EC2/EKS/ECS)         10.0.10.0/23  10.0.12.0/23  0.0.0.0/0 → NAT GW (same AZ)
  Private data (RDS, ElastiCache)   10.0.20.0/24  10.0.21.0/24  none (no internet)
Security groups: ALB-SG (443 from internet) → APP-SG (8080 from ALB-SG) → DB-SG (5432 from APP-SG)
```

Cloud design rules: plan **non-overlapping CIDRs** with on-premises and every other VPC (you'll connect them later); leave room to grow (a /16 per VPC, /20–/24 per subnet; EKS pods need many IPs); public subnets only for load balancers and NAT; apps and databases in private subnets; spread across **at least two AZs**; security groups referencing each other instead of IP ranges; VPC endpoints for S3/ECR to reduce NAT cost; Flow Logs on.

19.3 Hybrid connectivity

Need	Solution
Quick, encrypted, internet-based	Site-to-Site VPN (BGP over two tunnels for automatic failover)
Consistent bandwidth/latency, large data	Direct Connect (with a VPN as backup, or MACsec/IPsec over DX for encryption)
Many VPCs + on-premises	Transit Gateway (or Cloud WAN) as the hub
Name resolution both ways	Route 53 Resolver inbound/outbound endpoints and forwarding rules

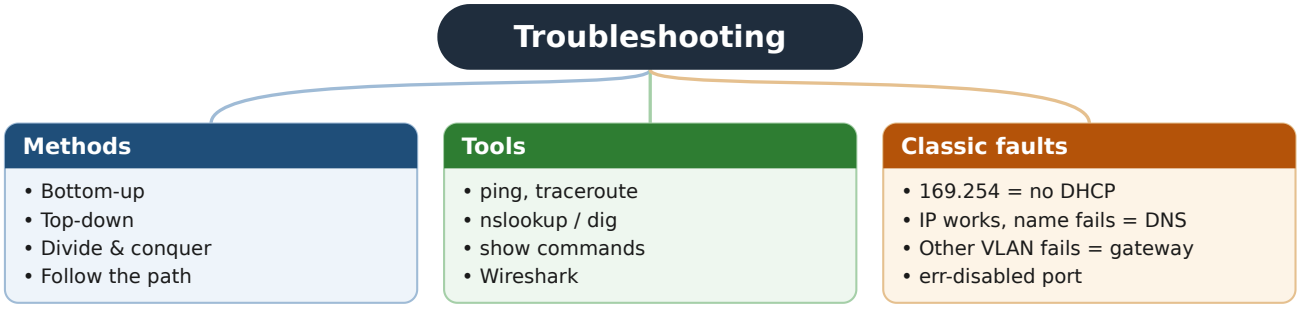
19.4 Kubernetes networking (for DevOps engineers)

Every pod gets its own IP (CNI plugin: AWS VPC CNI gives pods real VPC IPs; Calico, Cilium); **Services** give stable virtual IPs and DNS names; kube-proxy or eBPF programs the forwarding rules; **Ingress/Gateway API** handle HTTP routing from load balancers; **NetworkPolicies** are the firewall between pods; **CoreDNS** answers service names (`api.shop.svc.cluster.local`). See the Kubernetes notes for depth.

REMEMBER

- Plan **non-overlapping CIDRs** across on-premises and every VPC before you build.

20. Troubleshooting



20.1 Methods

Method	How	Best when
Bottom-up	Start at Layer 1 (cables, lights, interface status) and move up	New installations, physical problems suspected
Top-down	Start at the application and move down	Only one application fails
Divide and conquer	Start in the middle (e.g. `ping` = Layer 3) and go up or down depending on the result	Most situations: fastest in practice
Follow the path	Trace the packet hop by hop: host → switch → gateway → routers → server, **and back**	Routing/firewall issues
Compare configurations	Compare with a working device/site	Similar sites, after changes
Swap components	Replace cable, port, optic, device	Suspected hardware fault

Structured process: define the problem (who, what, since when) → gather information (`show` commands, logs, users) → **what changed?** → form a hypothesis → test it (one change at a time) → fix → **verify** → document and prevent.

20.2 Tools

Tool	Windows	Linux / macOS	Cisco IOS
IP config	`ipconfig /all`	`ip a` / `ifconfig`	`show ip interface brief`
Reachability	`ping` (`-t` continuous)	`ping`	`ping`, extended ping (`ping` + Enter for options: source interface, size, DF bit, repeat)
Path	`tracert`	`traceroute`, `mtr`	`traceroute`
ARP / neighbors	`arp -a`	`ip neigh`	`show ip arp`, `show mac address-table`
Routes	`route print`	`ip route`	`show ip route`
DNS	`nslookup`	`dig`, `nslookup`	`ping <name>` with `ip name-server` set

Tool	Windows	Linux / macOS	Cisco IOS
Ports	`netstat -ano`, `Test-NetConnection host -Port 443`	`ss -tulpn`, `nc -zv host 443`	`telnet <ip> <port>` (as a port test)
Packets	Wireshark	`tcpdump`, Wireshark	Embedded Packet Capture, **SPAN/RSPAN** port mirroring to a Wireshark PC

```
R1# ping
Protocol [ip]:
Target IP address: 10.2.2.20
Repeat count [5]: 100
Datagram size [100]: 1500
Extended commands [n]: y
Source address or interface: 10.1.1.1           ! test the path FROM the LAN, as users experience it
Set DF bit in IP header? [no]: yes              ! find MTU problems
```

Cisco ping symbols: `!` reply · `.` timeout · `U` destination unreachable (a router said no route) · `Q` source quench · `M` could not fragment (MTU) · `?` unknown packet type · `&` TTL exceeded.

Ping results on hosts: `Reply` = OK · `Request timed out` = no answer (host down, firewall, or routing on the **return** path) · `Destination host unreachable` = a router or your own host has no route / no ARP reply · `TTL expired in transit` = routing loop.

20.3 Common scenarios

Problem	Likely causes and checks
PC has 169.254.x.x	DHCP not reachable: cable/VLAN wrong, DHCP server down, missing `ip helper-address`, pool exhausted, DHCP snooping dropping (untrusted uplink)
Can ping by IP, not by name	DNS: wrong DNS server on the host, DNS server down, record missing (`nslookup`)
Same VLAN works, other VLANs don't	Host default gateway wrong/missing, SVI down or wrong IP, `ip routing` missing on the L3 switch, subinterface encapsulation VLAN wrong, trunk not allowing the VLAN
Two switches, a VLAN doesn't pass	Trunk not formed (DTP modes, `show interfaces trunk`), VLAN not created on both, VLAN not allowed, native VLAN mismatch
Port is err-disabled	Port security violation, BPDU Guard, EtherChannel misconfig → fix the cause, `shutdown` / `no shutdown`
Whole network slow / broadcast storm	Layer 2 loop (STP disabled, BPDU filter, unmanaged switch loop), duplex mismatch, congested uplink, interface errors
OSPF neighbors not forming	Subnet/mask, area, hello/dead timers, MTU, passive interface, duplicate router ID, authentication, ACL
Internet works by IP from the router but not from the LAN	NAT inside/outside reversed, NAT ACL doesn't match the LAN, no default route on internal L3 devices
Remote site unreachable	Missing route on one side (**return path**), VPN tunnel down, ACL/firewall, overlapping subnets
Intermittent drops	Interface errors (CRC), flapping link, Wi-Fi interference, duplicate IP address, overloaded device CPU

Problem	Likely causes and checks
Some websites fail, others work	MTU/MSS issue over tunnels/PPPoE, DNS, proxy, firewall/URL filtering
New IP phone gets no voice VLAN	Voice VLAN not configured, CDP/LLDP disabled, PoE budget exhausted

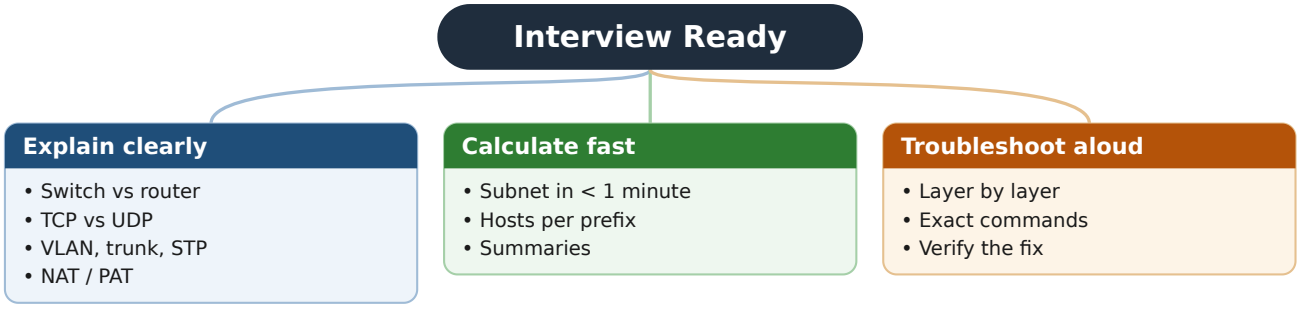
20.4 Layer-by-layer checklist

Layer	Check
1	Link lights, `show interfaces status`, errors/CRC, cable/optic, speed/duplex, PoE
2	VLAN membership, trunks and allowed VLANs, STP state (blocking?), MAC table, port security, EtherChannel status
3	IP/mask/gateway, `show ip route`, ARP, ACLs, NAT, routing protocol neighbors, ping/traceroute
4	Port open? Firewall rules, `telnet host port` / `Test-NetConnection`
7	DNS, application logs, certificates, proxy settings

REMEMBER

- Start in the middle with **ping** (Layer 3) and go up or down from the result.
- Process: define → gather → what changed? → hypothesis → test → fix → verify → document.

21. Interview & Exam Preparation



21.1 Interview questions (quick answers)

Question	Short answer
Switch vs router?	Switch forwards frames within a network by MAC address (L2); router forwards packets between networks by IP address (L3) and separates broadcast domains
What happens when you type a URL in a browser?	DNS lookup → TCP handshake → TLS handshake → HTTP request → server response → page rendered; each packet goes to the default gateway with ARP for its MAC
TCP vs UDP?	TCP: connection-oriented, reliable, ordered, flow control. UDP: connectionless, low overhead, no guarantees
What is ARP?	Resolves an IPv4 address to a MAC address on the local network using a broadcast request and unicast reply
What is a VLAN and why use it?	A logical Layer 2 network on switches; separate broadcast domains for security, performance and organization
Access vs trunk port?	Access: one VLAN, untagged, for end devices. Trunk: many VLANs with 802.1Q tags, between switches/routers
What is the native VLAN?	The VLAN sent untagged on a trunk; must match both ends; set it to an unused VLAN
Why STP? How is the root chosen?	Prevents Layer 2 loops with redundant links. Lowest bridge ID (priority, then MAC) wins
PortFast and BPDU Guard?	PortFast: access ports forward immediately. BPDU Guard: err-disables a PortFast port that receives a BPDU
What is EtherChannel?	Bundles links into one logical link for bandwidth and redundancy; LACP negotiates it
What is a default gateway?	The router a host uses to reach other networks
How does a router choose a route?	Longest prefix match, then administrative distance, then metric
Static vs dynamic routing?	Static: manual, simple, no adaptation. Dynamic: routers exchange routes and adapt to failures
OSPF vs BGP?	OSPF: link-state IGP inside one organization (cost metric, SPF). BGP: path-vector EGP between autonomous systems, policy-based

Question	Short answer
What is administrative distance?	Trustworthiness of a route source; lower wins (connected 0, static 1, OSPF 110)
OSPF neighbor requirements?	Same subnet, area, timers, authentication, MTU; unique router IDs; not passive
DR/BDR purpose and election?	Reduce adjacencies/flooding on multi-access networks; highest priority then highest router ID; non-preemptive
What is a floating static route?	A backup static route with a higher AD than the primary route
What is HSRP?	Cisco FHRP: routers share a virtual gateway IP/MAC; active/standby with priority and preemption
NAT vs PAT?	NAT maps IP addresses; PAT maps many private IPs to one public IP using source ports
Public vs private IP?	Public: routable on the internet. Private (RFC 1918: 10/8, 172.16/12, 192.168/16): internal only, needs NAT
Explain DHCP.	DORA: Discover, Offer, Request, Ack; relay with `ip helper-address` across subnets
Standard vs extended ACL?	Standard: source only, place near destination. Extended: protocol/source/destination/ports, place near source. Implicit deny at the end
Port security violation modes?	Shutdown (err-disable), restrict (drop + log), protect (drop silently)
DHCP snooping and DAI?	Snooping blocks rogue DHCP servers on untrusted ports and builds a binding table; DAI uses it to drop spoofed ARP
RADIUS vs TACACS+?	RADIUS: UDP, encrypts password only, network access. TACACS+: TCP 49, encrypts everything, per-command authorization for device admin
WPA2 vs WPA3?	WPA3 uses SAE instead of PSK (resists offline guessing), requires PMF; Enterprise modes use 802.1X with RADIUS
Lightweight AP vs autonomous AP?	Lightweight APs are managed by a WLC over CAPWAP; autonomous APs are configured individually
How many hosts in a /26?	64 addresses, 62 usable
IPv6 link-local address?	`fe80::/10`, automatic on every IPv6 interface, used for NDP and as routing next hop
What replaces ARP in IPv6?	Neighbor Discovery (ICMPv6 NS/NA)
SDN northbound vs southbound?	Northbound: controller ↔ applications (REST). Southbound: controller ↔ devices (NETCONF, RESTCONF, OpenFlow)
What is REST and CRUD?	HTTP-based API style; Create/Read/Update/Delete map to POST/GET/PUT-PATCH/DELETE
Ansible vs Puppet?	Ansible: agentless push over SSH/APIs (YAML). Puppet: agent-based pull
How do you troubleshoot "no internet"?	IP config → ping gateway → ping public IP → DNS → NAT/firewall/proxy → application
Security Group vs NACL (AWS)?	SG: stateful, instance level, allow only. NACL: stateless, subnet level, allow and deny
What is SD-WAN?	Centrally managed overlay WAN using several transports with application-aware path selection

Question	Short answer
What is a VPN?	An encrypted tunnel over an untrusted network (IPsec site-to-site/remote access, TLS)

21.2 Scenario questions to practice out loud

1. Users in VLAN 20 can't get IP addresses after a new switch was installed. Walk through your checks.
2. A new switch became STP root and traffic now takes a slow path. Why, and how do you prevent it?
3. Two routers are connected but OSPF stays in EXSTART. What's wrong?
4. Users can browse by IP but not by name. What do you check?
5. The branch can reach head office, but head office can't reach the branch. What's likely missing?
6. A user's port keeps going err-disabled. How do you find and fix the cause?
7. Design the IP plan (VLSM) for a site with 120 users, 40 phones, 20 servers, 2 WAN links using 10.10.0.0/22.
8. The primary internet link fails but traffic doesn't move to the backup. What could be wrong with the floating static route?

21.3 CCNA exam strategy

- Know the **verification commands** and how to read their output, not only configuration commands.
- **Subnetting must be fast** (under 30 seconds per question): practice daily.
- In simulations, read the whole task first, save the configuration if asked (`copy run start`), and verify every requirement with `show` commands.
- Watch for traps: implicit deny in ACLs, ACL direction, the native VLAN, `switchport trunk allowed vlan` without `add`, wildcard masks, OSPF non-preemptive DR, HSRP without `preempt`, AD of floating routes.
- Learn the CCNA v1.1 additions: AI/ML in network operations, cloud management, Terraform/Ansible concepts.
- You can't go back to previous questions in the Cisco exam: manage time (roughly 1 minute per question; simulations take longer).

21.4 Self-check

- [ ] I can explain every OSI layer and follow a packet across routers (which addresses change and which don't).
- [ ] I can choose cabling, optics and PoE for a given requirement and read interface errors.
- [ ] I can subnet any IPv4 address in under 30 seconds and design VLSM plans.
- [ ] I can write and shorten IPv6 addresses, build an EUI-64 interface ID, and explain SLAAC/DHCPv6.
- [ ] I can configure and verify VLANs, trunks, inter-VLAN routing, Rapid PVST+ with guard features, and LACP EtherChannel.
- [ ] I can read a routing table, apply longest-match/AD/metric, and configure static, floating static, IPv6 static and single-area OSPF.
- [ ] I can configure DHCP (server, relay, client), NAT/PAT, NTP, SNMPv3, syslog and explain QoS tools.
- [ ] I can explain wireless RF, architectures, AP modes, WLC connections and WPA2/WPA3 security, and configure a WLAN in the GUI.
- [ ] I can configure and verify ACLs, port security, DHCP snooping, DAI, SSH and AAA.
- [ ] I can explain SDN, REST APIs, JSON, YANG/NETCONF/RESTCONF, Ansible/Terraform and AI in network operations.
- [ ] I can map traditional networking to AWS VPC concepts and troubleshoot methodically.

REMEMBER

- Practise subnetting daily until it takes under a minute.